

Daily Notes - 2025 May

- 

May 30., Fri
Chaps
- 

May 29., Thu
Chaps
- 

May 28., Wed
Chaps Team
- 

May 27., Tue
Chaps
- 

May 26., Mon
Chaps
- 

May 23., Fri
Chaps Team
- 

May 22., Thu
Chaps Team
- 

May 21., Wed
Chaps Team
- 

May 20., Tue
Chaps Team
- 

May 19., Mon
Chaps
- 

May 16., Fri
Chaps Team
- 

May 15., Thu
Chaps
- 

May 14., Wed
Chaps Team
- 

May 13., Tue
Chaps Team
- 

May 12., Mon
Chaps
- 

May 9., Fri
Chaps Team
- 

May 8., Thu
Chaps Team
- 

May 7., Wed
Chaps Team
- 

May 6., Tue
Chaps
- 

May 5., Mon
Chaps Team



Chaps

Levy Melnikov · Thu, May 29 · Added profile & profile image APIs · Explored our options for model usage tracking ...

↑ May 30., Fri

Chaps



Levy Melnikov



2025. May 29., Thu

- Added profile & profile image APIs
- Explored our options for model usage tracking

Model usage tracking

```
Redis connected http://localhost:8080
Getting usage for chap ID: chap_f94e845a-d387-4de2-82fd-e915c14c5103 {
  "model": "openai/gpt-4.1",
  "completion_tokens": 193,
  "prompt_tokens": 1716
}
Chap ID: chap_f94e845a-d387-4de2-82fd-e915c14c5103, Usage: {
  {
    "model": "openai/gpt-4.1",
    "promptTokens": 16485,
    "completionTokens": 619
  },
  {
    "model": "openai/gpt-4.1",
    "promptTokens": 33454,
    "completionTokens": 1792
  }
}

> [!] run_command_safe
Searching all user task files for current open tasks
{"command": "grep '[^\\]\\|\\|\\|' /docs/*.md"}
> {"type": "tool_result", "tool": "run_command_safe", "result": "", "agent_id": "conductor_agent", "timestamp": "2025-05-29T16:35:52.604636"}
{"type": "end_tool_use", "content": "Tool usage complete", "agent_id": "conductor_agent", "timestamp": "2025-05-29T16:35:54.458253"}

> chap_json: {"icon": "list", "icon_emoji_fallback": "\ud83d\uddcdd", "title": "No Active Tasks Found", "body": {"type": "text", "text": "You currently have no active tasks in your list. If you\u2019d like to add a new task or set a reminder, just let me know!"}, "followup": {"note": "If you want to, I can help you add a new task or set up a reminder.", "buttons": [{"label": "Add a task", "action": {"type": "prompt", "prompt": "Add a new task"}}, {"label": "Set a reminder", "action": {"type": "prompt", "prompt": "Set a reminder"}}]}, "body_md": "You currently have no active tasks in your list. If you\u2019d like to add a new task or set a reminder, just let me know!"}
text: \ud83d\udcdd No Active Tasks Found
You currently have no active tasks in your list. If you\u2019d like to add a new task or set a reminder, just let me know!
If you want to, I can help you add a new task or set up a reminder.
```

E2E flow PoC. OpenAI wrapper sends all usage statistics to auth, auth aggregates and stores usage.

Early findings:

- We can use langsmith to report tracing data to three sources: langsmith itself, grafana for metrics and our backend for billing. I've managed to make langsmith emit opentelemetry metrics to a custom destination. We could have an opentelemetry collector that then forwards these metrics to all three destinations.
- I've built an alternative prototype that just wraps OpenAI library and sends usage straight to backend with ~50 lines of code.

- In terms of our simplicity, control and independence from 3-rd party libraries this is a superior solution.
- I've also built a simple token usage tracking on auth: endpoint receives usage and sums it to existing usage in redis.

Profile & Profile Image APIs

It's implemented and is out on prod: [integration guide](#)

 2025. May 30., Fri

Continue with usage tracking

- Usage tracking call is blocking → find a way to make it async
- Emit logs on top of http call and make grafana pick those up so that we also get observability
- Finalize auth & chaplet PRs
- Writeup on findings

Continuing with usage → look into limits implementation

Ad-hoc support if any issues pop up

Observability

All metrics are now pushed from Fly.io's Grafana to our central Grafana Cloud instance. Platform dashboard <https://chaps.grafana.net/d/a672592e-021/platform-api-realtime?orgId=1&from=now-6h&to=now&timezone=browser&var-job=&refresh=10s>

Fly.io machine pooling

Did some research on how to do this: <https://s-staff.craft.me/gltrdqpKtrfvRJ>

Bottom line: pooling, in the sense we thought it would work as per recommendation, does not work.

However, I've drafted a solution in the above doc. Problem we're trying to solve: when a user registers, a new VM is created on Fly.io (the "Chaplet") for that user. Creating a machine on Fly.io takes time - upwards of 45 seconds. The user experience is far from optimal, unless Balint starts to upsell them on the Chaps experience to gain some time.

On Fly.io, machines are tied to apps. The initial implementation was an app is created per user, which has one machine running. If we stay with this, keeping a pool of machines would only be possible per user, which does not achieve what we want.

The solution would be to move off the "one app, one machine for one user" concept to "one shared app, one machine in that app per user". A slight variation of this will need to be used once we scale above a couple of hundred users, where users are put into buckets (apps) once the previous bucket is full. This is due to Fly.io's limitations on the number of machines an app can have (specifics undisclosed, of course).

Implementation plan is in <https://s-staff.craft.me/gltrdqpKtrfvRJ>. The first part is to decouple variable injections from the supervisor to the chaplet, and moving them into machine metadata. **I finished this today.**

Next, we'll need to route requests based on machine ID rather than app ID by maintaining a LRU routing table on the supervisor, backed by Redis persistence for cache misses. This work would need to be done at some point anyway as we scale out.

Then we can manage the pools, by keeping machines in a stopped state ready to be started - this will cut down start times from ~45s to roughly 5s if my calculations are correct. When a user registers, one of these hot standbys gets tagged with the Chaps-specific metadata - which was previously built into the chaplet - (this is 200ms instead of a config change which triggers a rebuild, as machine config is immutable), and is started. Stopped machines do not cost us money.

Next steps

┆ Tomorrow I'm off.

Next week the work to pool machines into an app I'm anticipating will take ~2,5 full days of work to fully set up the pooling. Moving to shared apps is something that we would have needed to tackle at some point anyway, as Fly.io's APIs - by their own admission - are not built to handle thousands of entities in a collection. Even with the added complexity I think this is a worthwhile time investment.

 Peter Sipos

merged response format developments



feat(agent-response): Structured chap response format

#25 · Opened by Schipy · Merged by Schipy

Merged

Otherwise researched MCP authentication landscape and wrote up a proposal how we could do auth



MCP chap configuration & authentication

Auth patterns in MCP Servers · Token: There are MCP servers that require manual config anyway, e...



Last Edited 5/29/2025

Highlights:

- **the basic api token based MCP servers are still relevant:** many, even majority of servers work like this, including Notion, Todoist, Figma, Github, Trello. This pattern may even survive long term for complex products, as with OAuth it's harder to manage scope (in the oauth flow the app requests scope, not the user is configuring scope, and eg in static oauth app model, there's only one scope Chaps can request from a service for a user vs with apiKeys multiple Chaps of the user can connect to same service with different scope). With complex products like Notion, the way it works is that you can define different integrations in the product with detailed and different configured scopes, and then each has its apiKey.
- OAuth is emerging, but is quite a wild west for now, there are all kinds of solutions regarding who does it, the MCP servers have no uniform API in this respect. **We should build the OAuth support modular,** we should separate
 - **the oauth control logic** (oauth discovery, generating the final oauth request to be performed, access token refresh management), powered mostly by off the shelf 3rd party oauth solutions
 - **the actual user grant in a browser**, which needs to happen in mobile browser (or with a magic link on desktop browser, see later)
 - **and how the access token is actually applied.** Eg some studio MCP servers need the tokens set as env vars (they put on the Authorization themselves when they call the provider API), remote-mcp stores it in a config file, Orchestra MCP SSE needs the final Authorization header injected.
- We can further tune our OAuth flow with a magic link. We just offer to send the same preconfigured auth link over email, which the user can open on a desktop, where she is logged in already to the product. The only challenge is that the desktop browser will not have cookies set, so will not be able to call back to the chap, so we will need a special short living token or similar trick here. In practice, the user may not have a desktop computer available at hand, so this is a flow which could help situations, but not a silver bullet. In some cases the user just has to log in to the service on her mobile browser.

- Bonus: We should likely also do a **Zapier** Chap, they have MCP support and you can configure its scope nicely (can even request multiple MCP servers for yourself with different scopes). Basically it would open up the ability to chat with anything that can be integrated with Zapier, meaning the whole world (8000+ apps). Requires an effort to set up something this way, but could be very powerful (in terms of demos, virality etc). I remember once we said it's not so bad if the product has certain depth, because then an expert ecosystem can form around it, who make videos about how to do stuff etc. May not play together well with how we imagined Chaps to be super mainstream and simple 🤔, but anyway, worth to consider.

Tomorrow

will work on an OAuth PoC, and try make it work with Linear MCP, Google Calendar MCP

↑ Daily Notes - 2025 May

May 29., Thu



Chaps

Levy Melnikov · Wed, May 28 · Image Prompts · I've finalized everything related to securely storing thread assets an...

↑ May 29., Thu

Chaps



Levy Melnikov



2025. May 28., Wed

Image Prompts

I've finalized everything related to securely storing thread assets and making a use of them in LLMs

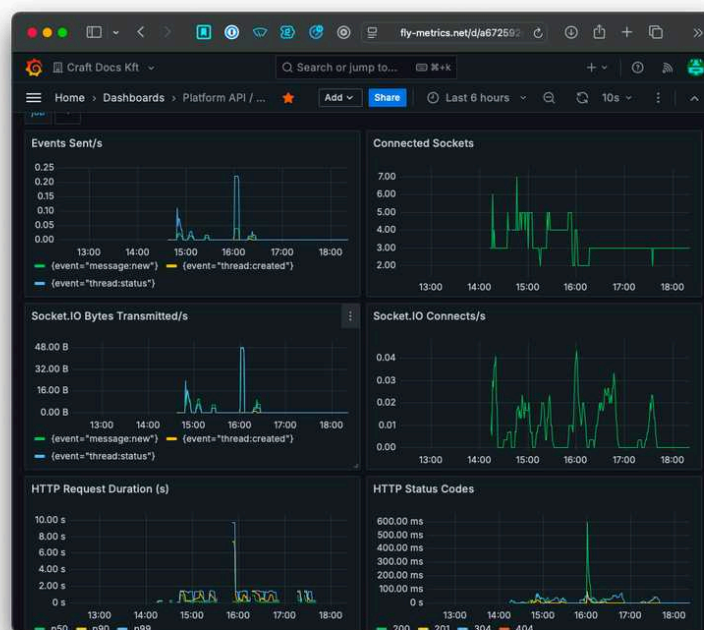
Auth

Implemented cookie-based auth for widgets. This is needed for when a Chap creates a widget and needs to serve it to the client. In this case, we can't use the Auth header to send the session token - it's read from the `_chaps_auth` cookie.

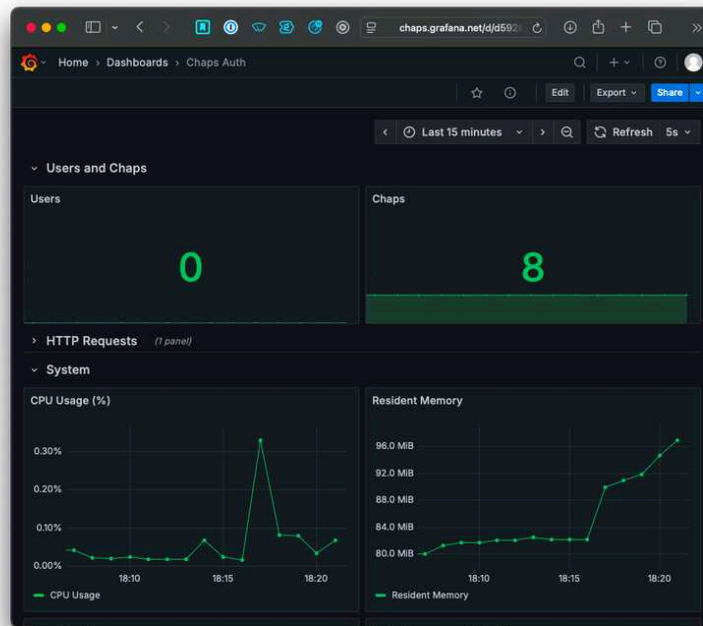
Observability

I've built observability and metrics around Prometheus. Prometheus scrapes an endpoint on services (`/metrics` for platform, `/api/v1/metrics` for auth), which is funnelled to Grafana for visualisation and alerts.

On Fly.io, Prometheus and Grafana are provided, and we can emit custom metrics there. For Platform, this was the fastest way to get metrics. I built a dashboard for Platform HTTP API and SocketIO socket stats <https://fly-metrics.net/d/a672592e-021/platform-api-realtime?orgId=1022432&refresh=10s&from=now-6h&to=now>



For Vercel (Auth), I needed to implement a different way to get metrics. I've set up our Grafana Cloud account (free tier), which provides a Prometheus importer. The importer scrapes `/api/v1/metrics` authenticated every 30s, and funnels metrics into Grafana. The tricky part with serverless metrics is HTTP requests need to be recorded by ourselves. I've also implemented 2 custom metrics - user count, chap count - we can easily add more later. The dashboard for auth is in a different place, on Grafana Cloud here <https://chaps.grafana.net/d/d5928ca6-8af4-49cd-8d98-ebd36a10939f/chaps-auth?orgId=1&from=now-15m&to=now&timezone=browser&refresh=5s>.



Note: User count is zero due to users not being counted before. This will increase once we have new (ie. not us) registrations, or we delete our accounts and re-register. I'll script it to update it to the correct number of users now if need be.

Some data still needs to be populated in these, but new metrics are being scraped.

Grafana Cloud can import metrics from Fly.io as well - I didn't get around to linking all of them up, I'll do the remainder tomorrow morning, that way dashboards are in the same place and Grafana Cloud is the single source of truth → [I've started moving them to this folder in Grafana Cloud](#). Platform API / sockets is missing some data there, view on Fly.io for now.

Grafana Cloud shared account login in 1Password:



1Password — Password Manager for Teams, Businesses, and Families

<https://start.1password.com/open/i?a=4NOBVX6KUBB3ZLMUNGUV4R6PV4&v=k7ymd2mmjwhqu3tk6m3anuf7ay&i=w4aj22ljgjpfix...>

Tomorrow

- Investigate Fly.io edge routing (see Sipi's note) *if long-polling is necessary*. Currently, everything works with websockets.
- Spoke to Sipi - we'll need to change what we store for messages. Currently we store a string 'message' for content, and there's a freeform metadata field - which is being used now. We need to support a JSON structure for message content parts - content: string, contentParts: JSON. I'll update the APIs.
- Anything else related to integration
- Observability - let's discuss with Levy

- Smaller observability tasks - moving dash to Grafana Cloud, custom metrics for logins in Auth (failed login attempts, etc.)
- Supervisor metrics. Straightforward integration based on  2025. May 28., Wed 's work.
- Chaplet metrics - let's discuss what custom ones we want to see, as we already have machine metrics in Fly.io's Grafana
- Pulling together these for one overview dashboard
- Setting up alerts to a Slack channel
- Start on: separate environments - sandbox, prod. Depending on how Levy's progressing we can split the work.

in the morning worked/supported a various topics

- investigating / debugging the connectivity issue we had in prod. conclusion:
long polling transport does not work atm in the fly.io infra (non sticky sessions somewhere in fly edge routing) → solution for now is to force websocket transport. We agreed with the guys that for now we go with this on the short term, to not cause additional infra changes while integration is in focus. We can revisit this later in upcoming weeks somewhere.
- we also discussed chap widget auth: I changed to urls to point to an injected var that is resolved to the supervisor in prod, where a cookie based auth has been added.

Bottom line (cc  Balint Orosz): the widget web view will need the session token injected as a `_chaps_auth` cookie (http-only), which will authenticate all requests coming from that view going to the supervisor, which will do auth and route based on user to the appropriate chaplet, where the gateway is already serving widgets.

- also discussed the reconnection fix, which Levy implemented

Then for the bigger part of the day I worked on the **response format of chaps**.

(I have it together now, **will merge tomorrow morning**)

TL;DR:

- `message.content` will be a markdown text version of the comms output
- and there will be a `message.metadata.chapJsonStr` which will contain the structured, richer JSON comms output (that can include widgets etc)

The changes in more detail:

The chap emits two versions of the content, a markdown text response, and a structured json response for rich content.

User: need to call Norbi

the markdown text response

```
2  📞 Task added: Call Norbi
3  - [ ] call Norbi
4
   If you want to, I can also help you set a reminder or add a deadline for this call.
```

- forms the message history from the chap's perspective, when conversation continues, (as it won't render widgets when convo continues and this message needs to be passed in to agent as history)
- this content is also structured logically (includes icon, title, body etc), and is created from the comms output, so contains verified actions etc

- it needs to work this way (to add the comms output to history), as user responses, including prompt recommendations the chap sends (see followup.button part of structured response) will refer to the message seen by the user, so the conversation will be only coherent this way.
- also serves as message input to other chap agents in the thread and serves as a general fallback or default for all headless clients (which won't render widgets when they receive a message), or clients that don't understand chap rich content.
- also useful for potential message indexing and search

the chap json output

```

1  [
2      {
3          "icon": "phone",
4          "icon_emoji_fallback": "📞",
5          "title": "Task added: Call Norbi",
6          "body":
7          {
8              "type": "widget-url",
9              "url": "http://localhost:3000/chaps/ef2a73dc-65bd-4c1c-af59-
10 b89eff2a7aea/widgets-common/task?
11 data=eyJ0aXRzZSI6ICJjYWxsIE5vcmlJpIiwgImNoZWNrZWQiOiBmYWxzZX0=&open=todo-list/tasks"
12 },
13          "followup":
14          {
15              "note": "If you want to, I can also help you set a reminder or add a
16              deadline for this call.",
17              "buttons":
18              [
19                  {
20                      "label": "Set a reminder",
21                      "action":
22                      {
23                          "type": "prompt",
24                          "prompt": "Remind me to call Norbi"
25                      }
26                  },
27                  {
28                      "label": "Add deadline",
29                      "action":
30                      {
31                          "type": "prompt",
32                          "prompt": "Set a deadline for calling Norbi"
33                      }
34                  }
35              ]
36          },
37          "body_md": "- [ ] call Norbi\n\n\n"
38      }
39  ]

```

- this contains the structured content sections that are used by Chaps clients to show rich content.

- discussed with BalintO briefly, and follow-up actions now take the format of prompt recommendations, when tapped, they prefill the chat with the given prompt. Naturally these prompts are dynamically generated based on the conversation and things happening.
- I ended up adding a `body_md` prop as well, which is a markdown fallback of the widget. Practically it is the same as the body part of the text response, but figured it allows a client that otherwise renders the rich content to fall back just for the section body part in case the widget has issues or is unknown etc. (but still render actions, the lucide icon etc and not the pure markdown version). We can play with whether this should be a markdown or plain text fallback here.
- If not widgets are used in the response, then the `body` is also a markdown text (same as the fallback). In this case the markdown has some significance as it gives some structure and flare to the otherwise plain message. (not showcased in next example, but sometimes)

```

1  {
2    "icon": "cup-soda",
3    "icon_emoji_fallback": "🥤",
4    "title": "Preference recorded",
5    "body":
6    {
7      "type": "text",
8      "text": "I've noted your preference for always drinking lactose free milk.
9      This will be kept in mind for any related tasks or reminders."
10   },
11   "followup": "...",
12   "body_md": "I've noted your preference for always drinking lactose free milk.
13   This will be kept in mind for any related tasks or reminders."
14 }

```

An interesting aspect is that these outputs are part of the widget definitions, so the system is extensible with future widgets. The way it works is that a widget:

- defines its JSON format in the form of LLM instructions
 - (this JSON goes into the general comms JSON format)
 - (and we also turn on json mode now for comms agent)
- the widget also defines a `to_markdown` logic as a `jinja2` template (THE template lib for Python, like weekly 70M downloads). Eg for todos:

```

1  to_markdown: |-
2    {% for item in items %}
3    - [{% if item.checked %}x{% else %} {% endif %}] [{{ item.title }} {% if
4    item.deadline %}{{ item.deadline[:16] }}{% endif %}
5    {% endfor %}

```

- using this `to_markdown` we can create the markdown mapping of the structured output (which is the text response of the chat message emitted to thread)
- the widget also defines the web app path that can render the output data, so we also programatically convert the section body to widget format for the final structured output (where the data gets encoded into b64 and passed to the web app as a `data` query param)

Tomorrow

bring this branch 🙌 home

one next thing that I have in mind is working a bit on the chap dynamic configuration, this aims at

- supporting chap setup (esp. MCP) via a full screen web widget without making Postman calls (though could be a native UI as well). We need some UI somewhere, as even staff testers we don't want to force to make Postman calls to be able to use Chaps. An option is to have a (styleable) common chap config widget. (think that chap defines its config schema, and we generate a simple config UI from it, limited expressive power only of course),
- also moves toward the generic MCP chap (as that needs not just an apiKey)

also maybe extracting a proper base chap agent/prompt config

- which works towards being able to add more chap types faster (now we still have a bunch of dup in chap configs.yaml-s)

Also if anything comes up in app integration, that is a priority.

I can also look into some other specific widgets (eg a Notion page), or look into how a simple callout fits into the picture.

↑ Daily Notes - 2025 May

May 28., Wed



Chaps Team

Peter Sipos · various plumbing around widgets, without going into too much detail, what I have now · a web app whi...

↑ May 28., Wed

Chaps Team

various plumbing around widgets, without going into too much detail, what I have now

- a web app which hosts common widgets, which is for now an `inline todo-list`, and a `single todo/task view` (their design is just a draft/PoC)
- if a chap bundles web apps, their inline widget configs are automatically mixed into the chap config, and they can be then exposed for comms agents with a one liner, like

```
1 You can use the following section types:
2   ${widgets.task_item.comm_format}
3   ${widgets.todo_list.comm_format}
```

- we also have two chap specific todo list web apps, which are intended to be interactive fullscreen views
 - one for the todo-chap that works based on file operations
 - and one for todoist, that works based on todoist api calls
 - these are bundled into their chaps respectively (along with common widgets)
- the full screen todo-list url is passed to the inline widgets as an open action, which we can wire in so that it opens on tap on the widget

How to spin up a meaningful chap now

General

- try now `todo-chap` or `todoist`, notion-mcp is a bit behind
- for MCP based chaps, after it is created, you need to set the `apiKey` as runtime/instance configuration. This can be done via [this endpoint](#).
basically it's a POST `/chaps/:chapId/appconfig/api/configure`, form encoded, with an `apiKey` value

In prod

- create a chap with new `/api/v1/chap` endpoint (type: `todo-chap` or `todoist`)
- set `apiKey` with endpoint above
- create thread with chap as participant, send a message

Running chap locally (fallback)

- have Docker desktop app running
- clone & setup `chaps-api` repo then run chat platform locally
 - (`bun install` & `bun start` in repo root)
 - this does not need docker, just runs locally
- clone & setup `chaplet` repo, and then `./dev.sh` in repo root
 - this will run the chaplet in Docker
- create a chap either via chaplet's [create chap api](#) (in local env)
 - or via the CLI tool (in `./tools/gw` `bun install` & `bun start`)

- the chap will connect to local platform and receives sends messages there
- can send/receive messages on *Platform APIs* (pointing to local env - localhost:3001)

 Levy Melnikov

 2025. May 27., Tue

I've worked on the ability of a Chap to understand images

```
(use "git add <file>..." to include in what will be committed)
dish.jpg

no changes added to commit (use "git add" and/or "git commit -a")
levymelnikov@Levys gw % bun start
  bun run clerk:build && bun src/cli.tsx
  vite build clerk
vite v6.2.4 building for production...
✓ 23 modules transformed.
[plugin vite:singlefile]
[plugin vite:singlefile] Inlining: index-D3R6D7fv.js
[plugin vite:singlefile] Inlining: style-Dqs120NQ.css
dist/index.html 1,860.96 kB | gzip: 628.64 kB
✓ built in 1.38s

Chat with Untitled Chap (@c22a3cc-ba17-4f16-94d8-e6587575e7ea)
> 
Type /f to browse files, /si to send an image, /q to quit, /u to manage your account, /s to switch chap, /m to manage your email
```

Todo chap creates a shopping list from an image of a dish(Skip to the end once it starts "Generating...")

Important bits

Orchestra library didn't provide an image to LLMs during tool loop. This caused the conductor to hallucinate adding ingredients of a dish that it didn't see and then communicator was gaslighting that everything was done correctly. I've [changed](#) our fork so that image is provided for tool loop. With this change this flow works nicely!

We are storing images as secure links. Just before they are passed to orchestra they are downloaded and base64 encoded.

Orchestra library doesn't allow prompts to have images, it expects them to be passed in a separate field. Current approach is to comply - prompt is stripped to be text only, images are provided to a dedicated parameter.

Technical bits



Image prompts

I have the PR but a bit more things are required before I can consider this done:

- Send message platform API doesn't support adding links

- I haven't tested this via platform. I'm currently testing via `gw` and I'm afraid that it could not work via platform. The problem I encountered is that when prompt contains images urls orchestra breaks. I fixed this only for prompt, but not for the whole history. I'm not sure if having an image url in history would also break it. I'm don't exactly know how history works. There's a json file on the chaplet, but useragent can also send history from the thread and I'm not sure how it ties together.
- The `gw` code is hacky. An asset can only be uploaded to a thread, but `gw` doesn't know anything about platform or threads. Current solution is that I create a new thread for each asset and add a chap there. Threads are created with `_uid=1` .
- There's no documentation for "thread assets" implementation

 2025. May 28., Wed

Images

- Add support for images to Send Message API
- Test solution via platform
- Add docs for "thread assets" implementation

Client development support

Client implementation is crucial for the state that we want to have at the end of the week. If there's any fixes needed for Balint, I'd also be happy to help with those.

Observability

Start looking into observability

Chap CR/D

Chaps now receive and persist a platform token correctly which the Agent SDK can use to authenticate.

Fixed wiring for Auth → Supervisor for Chap creation / deletion. **Note** : there's a lot of edge cases we still need to tackle here. Happy path is working. An example:

- Deletion of Chap which is in a thread with
 - The current user. Delete the thread?
 - Another user (shared thread). Remove Chap?

We'll need to work through these from a product and technical perspective.

Chap CRUD is now in Auth. Do not create Chaps via `api.chaps.app`. See [Postman docs](#).

Chaplet creation

A well-known problem with new Chaplets is the time it takes for them to spin up. I've built a new flow to create Chaplets. It involves changes from the client side as well for it to offer a good UX.

The client flow, all authenticated calls with the session token, is:

- **POST `api.chaps.app/supervisor/api/chaplet`**

This call initiates the creation of a Chaplet. If the Chaplet exists, it will return `ready: true`. Do not call if you are sure the user is already registered, as this will call Fly.io's API to verify the Chaplet's status, which adds latency. Return is:

```
1 {
2   "success": true,
3   "ready": false
4 }
```

This means the Chaplet creation process has started.

- **GET `api.chaps.app/supervisor/api/chaplet`**. Poll this endpoint for the Chaplet status. For planning purposes: a typical start time for a new Fly.io VM is **~40sec**. This will return a status object:

```
1 {
2   "ready": false,
3   "details": {
4     "app": true,
5     "chaplet": false
6   }
7 }
```

`ready: true` means the Chaplet is running. If `details.chaplet` is `false`, it's being created.

- After this, the Chaplet is ready, and Chaps are ready to be created.
- See [Postman docs](#).

To create Chaps, use the endpoints mentioned above.

Chaplet local dev

Modified Chaplet code to persist volumes across restarts locally for faster dev.

Auth rate-limiting

With Chap handling in Auth it's time for rate limits, as Chap creation can DoS a Chaplet.

Important:

- **Rate limits are off by default.** They can be set by env vars for rate limited endpoints. We'll discuss sane limits tomorrow.
- Rate limits can be bypassed by setting the `X-CHAPS-RATELIMIT-BYPASS` header. We can remove this later if necessary.

Rate limits have been set up (mostly per user) for:

- **Verify TOTP.** Endpoint could be easily used for enumeration of 4 digit code. Limited by phone number and IP.
- **Create Chap.** Easy DoS of Chaplets.
- **User find by phone number** (address book). Enumeration of users.
- **Login attempt (send TOTP).** Already implemented in a different way, will be migrated here.

Implemented with the `@upstash/ratelimit` package. This is switchable. I went with this library as it's edge compatible (Vercel functions use HTTPS for Redis) and provides analytics out-of-the-box: see https://console.upstash.com/ratelimit?teamid=a93bfa08-b7fd-4711-aef5-8c7068ed8ffc&prefix=ratelimit%3ARL_POST_USER_SEARCH.

Tomorrow

- I'll provide support to Levy to integrate the asset handling into the beginning of the pipeline (sending messages)
- Starting observability, sync with Levy
- Any fixes needed as Balint starts implementation

↑ Daily Notes - 2025 May

May 27., Tue



Chaps

Welcome Levy! · Balint Bartfai · Planning today and working with Levy to get him up to speed. · Chap auth · We'r...

↑ May 27., Tue

Chaps



Welcome Levy!

Planning today and working with Levy to get him up to speed.

Chap auth

We're moving all Chap creation to the Auth service. Chaps need to be able to authenticate themselves, which means they need tokens to be able to call the APIs - these tokens are generated on Auth, which are then passed to the Chaplet's Agent SDK on creation. This means Auth needs to handle Chap creation via an authenticated (user session token) call to create a Chap.

Implemented the Auth side of Chap creation, including token handling and Chap list / delete / profile on Auth. See [postman docs](#). There is one important piece missing which I will need to solve tomorrow, [see below](#) 📌

Currently, these session tokens are valid for 10 years so we don't need to start worrying about refreshing them. Additionally, in contrast to user session tokens, I'm saving them in Redis so if we need to recreate a Chap during the beta period, we can just pull the token and send it to the Chaplet.

Technical details:

- Each session token contains the identity (`userId` field in `sub`) of the caller, along with a `loginId` and `loginType`.
 - `loginType` is the channel the original auth token was acquired with. For now, users use `phone`.
 - `loginId` is the identifier of that channel. Currently this is the phone number itself.
- Chap tokens use the same structure, the key differences are
 - `userId` - the identity - is their Chap ID
 - `loginType` is chap, `loginId` is currently set to the Chap ID, but I'm thinking it would make more sense to set it as the initiating user ID (will flip this over tomorrow)
- Users initiate Chap creation on Auth, a Chap token is generated (on Auth - previously supervisor API handled this), the chaplet API is called from Auth with the token, and a Chap is created
- Agent SDK saves the received token to `data/.platformtoken` in the FS, and uses it to make subsequent API calls
 - **Levy implemented the Agent SDK side today**

This works nicely for existing Chaplets, as the VM itself does not need to be created on Fly.io. However, if the user is new and wishes to create their first Chap, the request times out (machine creation on Fly.io takes a long time, HTTP request times out). It's a terrible user experience after registering on Chaps.

To solve this, I'll be working on a way for the client to:

- initiate the creation of the Chaplet before the user creates their first Chap, with an immediate response once the Fly.io API call is starting - chaplet creation endpoint
- know if the VM is ready or not, somehow pulling this info from Fly.io's APIs - chaplet status endpoint

We can move Chap creation to call the API to start, and then event push via Platform API, which would mean

- setting up endpoints on Platform API to signal to the user their Chap is ready (which is called by the orchestrator)
- pushing that event to connected sockets with that user ID

However, I think there's more complexity and time to wire it all up with the sockets, so **I'll be working on a simple status endpoint** which the client can poll. In the long run, it would be very nice to get that message for all connected users that a new Chap has been created by them.

Tomorrow

- Support Levy where I need to
- Solve long machine creation problem detailed above
- I'd like to return to the Chaplet a bit to ensure Chaps are properly synced. Last time I checked that side of the fence was before my leave - apparently updating the Chaplet image discards the content, which should not be happening
- Implement and deploy simple rate limits for expensive endpoints ([see MVP here](#)) - we'll need this before beta
- Depending on how much time we have and how the asset work is progressing, I'd like to start on observability with Levy

 Levy Melnikov

 2025. May 26., Mon

Implemented uploading assets to threads and securely accessing them

Implemented chaplet auth token handling

Secure assets in threads

We now have an API for uploading assets to threads. To upload an image to a thread you to be a participant. You can request an upload link with the new `/api/v1/threads/:thread_id:/upload_asset_url` (also in Postman) API.

Uploaded image is available via: `https://thread-assets.chaps.app/v1/thread/<thread_id>/<id>`. R2 bucket is private, assets are served via a CloudFlare worker. That worker also does authorization. To access the asset you have to:

- Be a participant of a thread
- Provide `Authorization: Bearer ...` header with your token

I'm also leaving ability to "force" userId via `_uid` query parameter as we do in other places. It's behind feature flag, once we get to sandbox \ prod envs we can disable it on prod. Examples:

- https://thread-assets.chaps.app/v1/thread/thread_808c469c-d949-4a82-a565-c0c13ba1cf76/82495a53-3d89-461f-b311-ce580c1739e4?_uid=cu1_ea328c69-b5b5-48e2-816b-ebdda6a14b6e → My userId is used, you should see a fridge
- https://thread-assets.chaps.app/v1/thread/thread_808c469c-d949-4a82-a565-c0c13ba1cf76/82495a53-3d89-461f-b311-ce580c1739e4 → No auth, should deny

Assets are cached on edge. Authorization is not cached → we make request to the platform each time user requests an asset.

Chaplet Auth Token

When chaplet receives `create` request it now accepts two parameters: token and chapId. Both are optional. When chapId is provided it will be used instead of generating a random one. When token is provided it's saved to the chaplet and is used for all calls \ socket connections with platform on behalf of that chap. I couldn't test it properly, but hopefully it works.

 2025. May 27., Tue

Questions

Secure assets in threads

- It would be great to try out new system with some practical test → e.g. put image of dish into a message and ask todo Chap to add ingredients to the tasklist. Or maybe we should do it together? Or maybe I should do it?
- Do we want to cache authorization? Currently each asset request makes a request to the backend for auth. We can cache who's a member of which thread and thus reduce amount of backend requests. Essentially after downloading 1 image from a thread with auth all subsequent requests within 1 hour(configurable) will bypass auth by using cache.
- Currently thread assets are not tracked: you can upload whatever number of images you like and not put them into any thread \ message. Do we want to keep track of these? For example do we need an endpoint like: "Show me all assets that I uploaded?"
 - Do we need limits on number of uploaded images?

Chaplet Auth Token

- It would be great to verify Chaplet Auth token handling e2e.

Observability

I'd like to start on observability.

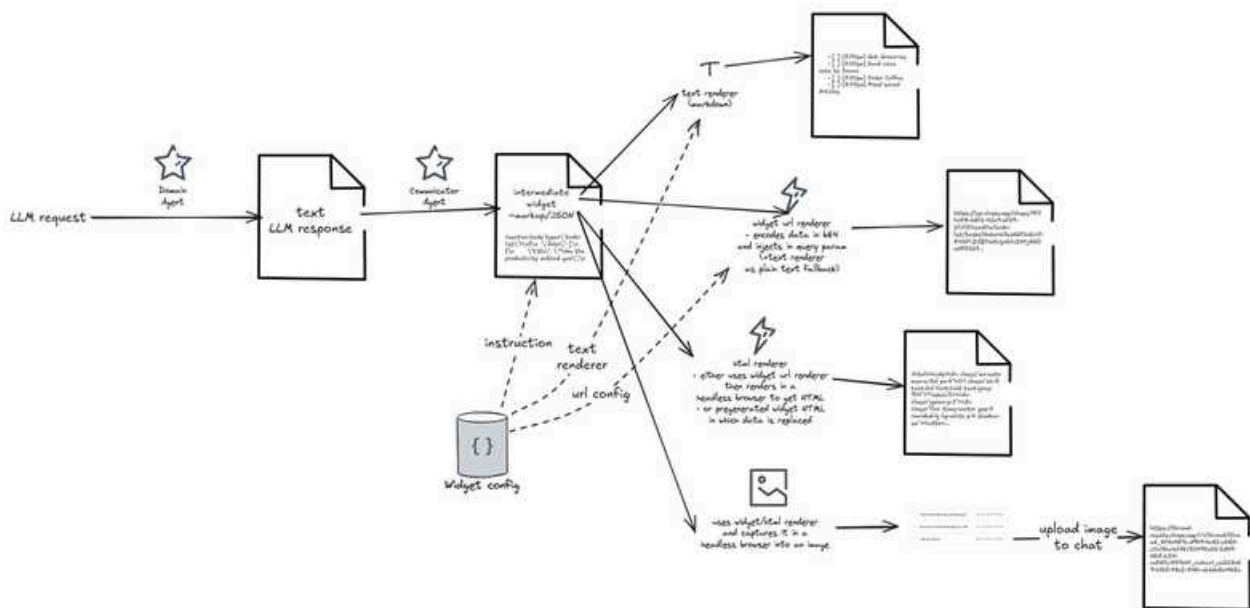
merged [agent configuration enhancements](#) (already used some of the extra flexibility today)

integrated chat platform `status` messages, a chap now

- sets an initial "Thinking..." status when it starts processing
- then sets tool call summaries (we'll be able to change the style/format of these via prompting)
- and then clears the status when processing finishes

Widgets

have an initial concept of how widgets would work (a PoC already works)



The main points

we generate an intermediate widget representation with the comms LLM agent. This format:

- encapsulates the data to be presented (as a generated JSON object practically)
- for interactive widgets, it can also contain not just the data, but the actions to be taken in the form of API calls.

a new widget *renderer* step converts this format into actually presentable widget formats, which can be:

- plain `text` (we can always generate this as fallback)
- **have this working** a `widget url` which gets the data in query param b64 encoded
- based on the widget url, a headless browser can render it to an `image`
- [later] based on the widget a headless browser can render `inline html` (or use some other templating)

- [even later] an abstract `component library description`, which is translated to `native` UI on app

I already managed to *decouple much of this widget logic from the actual chap*. This means it will not be hard to attach widgets to different chaps, eg the `todo-list` widget will be something that can be mixed into the config of both the todoist chap, our own todo chap, but also Notion etc. (new `config` merge for the win here again). Maybe different chaps will apply some of their own CSS to the widget.

Where I need input now (cc Balint Orosz):

1) a started ideating around widget use cases, and **I need some product input, which one would be interesting**

- `PoC working` non-interactive todo list
- interactive todo list (raises some flow questions)
- DB changes widget (eg for Notion)
- DB changes widget with some extra chap smartness (agent autofills details with web browsing)
- Reschedule a todo/reminder with calendar pick (raises many flow questions)
- also open to ideas about others (a chart or map would be quite cool)

see details inside 



Potential widget use cases to target

Todo-lists · Chap: Todoist / Notion / own todo chap · User: what is in my inbox? · Chap: Your Inbox Tasks · Image ...

2) widget size

the typical game when embedding webviews, how will we know the size (height)?

I'm thinking we could try to be dead simple here now, and it would be deterministic height based on the data? eg agent sends

```
1 <section-body type=\"widget-url\" height=\"264\">/*url*/</section-body>
```

one downside of lack of dynamic size negotiation is that we cannot do eg "click to expand" interactivity (progressive disclosure)

Details: The flow with example

The communicator agent gets a list of available widget instructions as potential section templates it can use, for example it generates:

```
1 <section-body type=\"todo-list\">
2 {
3   "data":
4   [
5     {
6       "title": "Take the productivity method quiz",
7       "checked": false,
8       "deadline": "2024-11-23T00:00:00.000Z"
9     }
```

```

10     },
11     {
12         "title": "Browse the Todoist Inspiration Hub",
13         "checked": false,
14         "deadline": "2024-11-24T00:00:00.000Z"
15     },
16     {
17         "title": "Call my dentist",
18         "checked": false,
19         "deadline": "2025-05-21T00:00:00.000Z"
20     }
21 ]
22 }
23 </section-body>

```

▼ the widget comm config for 🙌

```

1  widgets:
2    todo_list:
3      comm_format: |-
4        Todo section, JSON (described here in a pseudo-json format): use this if
5        todo/task items were involved ("type": "todo-list")
6        {
7          "data": [
8            {
9              "title": string;
10             "checked": boolean;
11             "deadline": string; // optional, ISO string format YYYY-MM-DDTHH:MM
12           }
13         ]
14       }
15       # ...
16       # in comm agent prompt
17       You can use the following section types:
18       ${widgets.todo_list.comm_format}
19       ${widgets.freeform.comm_format}

```

this then gets rendered based on again the widgets render config:

```

1  widgets:
2    todo_list:
3      renderer:
4        type: widget-url-dataquery
5        path: "tasks"

```

so it becomes eg

```

1  <section-body type=\"widget-url\">
2  http://api.chaps.app/chaps/:chapId/todo-list/tasks?data=W3sidG10bGUiOiAiVGFr...
3  </section-body>

```

Note: this is still "internal" representation in the sense that it's a text output with a special format. We can convert this in the agent client to a more structured message when it's emitted as a chat message, like

```
1 sections: [{
2   "type": "widget-url"
3   "icon": "inbox",
4   "title": "Your Inbox Tasks",
5   "url": "https://..."
6   "fallbackText": "- [ ] Take the productivity method quiz..."
7 }]
```

the widget web app parses the query param and uses that to populate the UI (a list of todo items in this case)

☐ Take the productivity method quiz Nov 23, 2024, 1:00 AM

☐ Browse the Todoist Inspiration Hub Nov 24, 2024, 1:00 AM

☐ Call my dentist May 21, 2025, 2:00 AM

↑ May 27, Tue · Chaps

Potential widget use cases to target

Todo-lists

Chap: Todoist / Notion / own todo chap

User: what is in my inbox?

Chap: Your Inbox Tasks

☐ Take the productivity method quiz Nov 23, 2024, 1:00 AM

☐ Browse the Todoist Inspiration Hub Nov 24, 2024, 1:00 AM

☐ Call my dentist May 21, 2025, 2:00 AM

<button>Open Todoist Chap</button>

If you want to I can also help you with...

<button>Mark as done</button> <button>Reschedule</button> <button>Edit task</button>

Advanced version: the checkboxes are interactive

List/DB changes summary

also showcase some add-on chap smartness?

Chap: Notion

User: Add movie to watch: Rain with a chance of meatballs

Chap: Added movie

<widget>

Movie Watchlist | 15 items /* imagine a nice card, where this is the heading*/

+ added: Rain with a chance of meatballs

</widget>

<button>Open Notion Chap</button>

If you want to I can also help you with...

<button>Mark as seen</button> <button>Add why you wanted to watch</button>

Advanced version:

Agent returns + added: Rain with a chance of meatballs, 2009, IMDB: 6.9

- imagine is it in chap's `preferences` already that agent should fetch some IMDB stats when working with movie watchlist, so a bit of extra chap smartness here. This also needs web search tools (orchestra has some, but we did not play with them much)

Efficient reminder schedule for a todo

User: I need to call my dentist

Chap: Added 1 todo

☐ Call my dentist

May 21, 2025, 2:00 AM

`<button>Open Todoist Chap</button>`

If you want to I can also help you with...

`<button>Mark as done</button> <button>Reschedule</button> <button>Edit task</button>`

User: taps on Reschedule

a full screen date picker comes up, when user picks a date, it closes

Chap: Updated reminder

☐ Call my dentist

May 21, 2025, 2:00 AM

`<button>Open Todoist Chap</button>`

If you want to I can also help you with...

`<button>Mark as done</button> <button>Reschedule</button> <button>Edit task</button>`

- this use case requires logic behind the buttons, not inside the widget
- calendar picker would be native? or a fulls screen widget?
- also it is not clear if we want an LLM run here, or an LLM-less API call

LLM call:

- PRO: robust execution with error recovery (achieves reschedule with MCP api sooner or later)
- CON: slower
- ?: yields a same (but maybe bit redundant?) user feedback

plain API call:

- PRO: fast
- CON: one shot attempt to reschedule with pre-recorded MCP call, if that fails then there's no error recovery. Maybe can fall back to LLM if API call fails?
- ?: how would user feedback work? (widget will not autoupdate, unless it's the one that makes the call in the end) Or alternatively we allow agent to post to thread without LLM calls? 🤔

↑ Daily Notes - 2025 May

May 26., Mon



Chaps

Balint Bartfai · Wrote up focus areas for Levy to work on from next week in Chaps: · Levy in Chaps · I tried to put the...

↑ May 26., Mon

Chaps



Balint Bartfai

Wrote up focus areas for Levy to work on from next week in Chaps:



Levy in Chaps

Welcome Levy! · We've put together some areas we need help with. Please reach out to BalintB for...



Last Edited 5/23/2025

📌 I tried to put these in priority order, but if something is more important from a product perspective feel free to reorder!

Platform API

Status

API has new endpoints for status updates in threads. See [Postman docs](#).

Status is an ephemeral state of a participant in a thread. Status is used for, example, typing indicators, thinking messages, etc. Each participant can independently set their status, a threads status (all participants) can be pulled from the API, individual participants can clear their status.

Under the hood, Status is a JSON object. This is stored in the servers memory, with an option to store in Redis if we scale out. It's not persisted to DB.

Threads will receive a `thread:status` event when any participants' status changes.

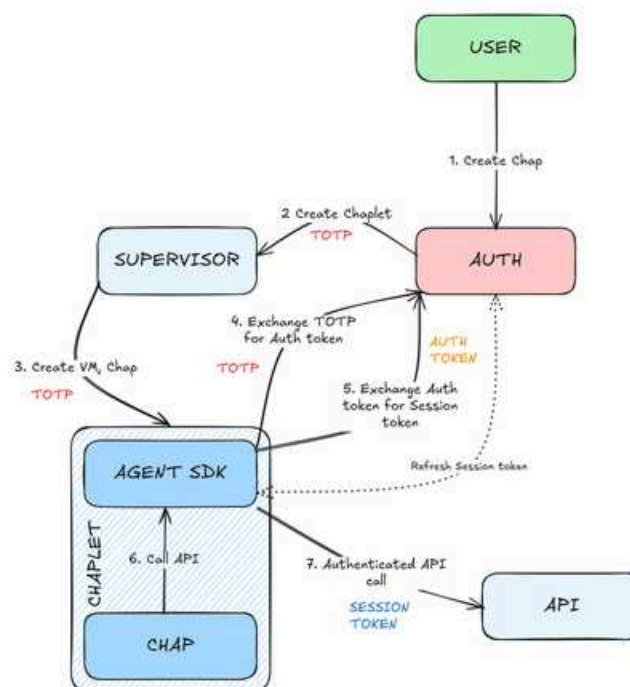
Address book

Auth has new endpoints for an "address book" functionality. Users can be searched for by phone number. Additionally, all users can be listed **for dev purposes** only - right now listing is an expensive operation. If we need to do this in prod I'll add proper fast support for it.

See [Postman docs](#).

Auth for Chaps

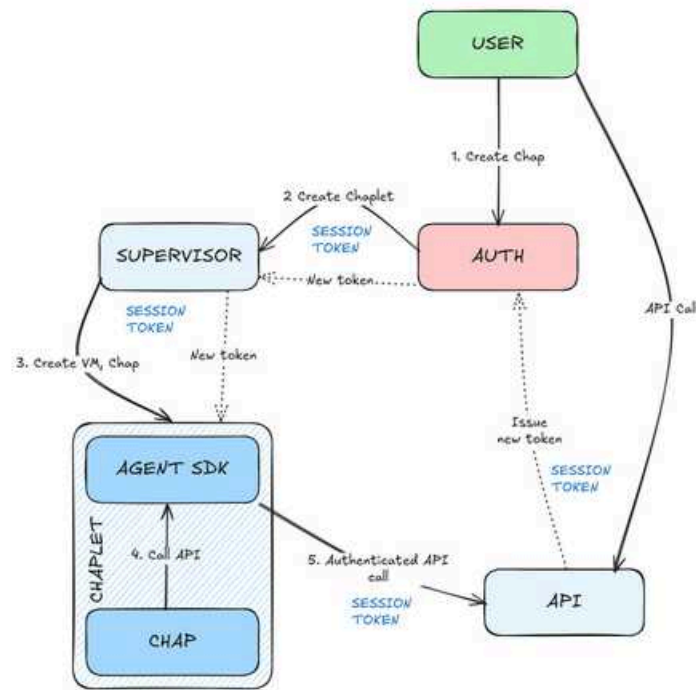
Chaps don't have a phone, so can't start auth by themselves with TOTP. We can still use the same principle for them to authenticate, by changing the beginning of the auth pipeline. Instead of a TOTP, new Chaps will receive a one time token on creation - this is created with an API call on behalf of an authenticated user. With that one time token, they start the same process as an app user would - they exchange it for a long-lived auth token and with that get a session token, which is valid on the APIs. The "ideal" process is shown below:



But this would require a lot of work upfront.

Simpler version we'll start with

For simplicity, we will issue API keys to Chaps directly on creation, which the agent SDK will handle. Keys will be issued every N days via the supervisor, on Agent or User API call (we could even implement heartbeats - see Agent SDK below). The simpler version of the above is:



I have started implementation on the Auth side. ETA ~3 days. For the Chaplet side, I think this would be a good warmup task for Levy to get to know how the Chaplet works and get into the Chaplet codebase.

Next steps

- Levy sync and architecture
- Will be continuing Auth for Chaps

 Peter Sipos

- [Last Friday's note](#) (was ooo on Friday)
- planning to work this Friday (will confirm by Monday)

UPDATE : Over the weekend I overhauled the chap configuration format to be simpler yet more powerful.

Summary:

instead of a bunch custom code I added in the past around the agent `config.json` (own interpolation logic to reuse prompt snippets and inject some configs, faking multiline strings) **we will now use a standard, simple and powerful lib called [OmageConf](#) for templating in the config** (5M weekly downloads - [stats](#)). It will be much simpler to maintain and extend, and will be flexible to support all kinds of use cases & tricks we might want to do with the agent configs (see below)

also switched from JSON to **YAML**, as YAML natively supports multiline strings & comments, it seems to be more convenient to manage and is widely adopted for configs. (it's a two way door change though, as omegaconf also works if config is serialized to json)

Goodies that will come:

- during agent development we'll be able to create chaps whose prompts we can edit on the fly (or any other config parts), without restarting or recreating the chap, reducing the prompt testing iteration time to practically 0. (test a message exchange → update prompt snippet in the chap via REST API / UI → test a message exchange with new prompt)
- also the whole config will be easier to manage, eg
can extract variables

```
1 default_llm: OpenrouterModels.custom_model(openai/gpt-4.1)
2 # later
3   agents:
4     communicator_agent:
5       llm: ${default_llm}
6       # could even do
7       llm: ${oc.select:request.flags.llm_override, default_llm}
```

and we can use a uniform syntax for various features for which we now have custom logic, eg instead of

```
1 agents:
2   todo_agent:
3     # some agent system prompt, blah-blah
4     # then current date will be injected hardcoded
5     # context files will be injected based on our auto_context_files (custom)
6     prop
7   auto_context_files: ["*.md"]
```

we can do something like

```
1 agents:
2   todo_agent:
3     # some agent system prompt, blah-blah
4     The current date is ${env.date}
5     # generalize preloaded files as prerun tools and inject result
6     ${run_tool: {name: "ChapTools.run_command", args: ["cat preferences.md"]}}
7     ${run_tool: {name: "ChapTools.run_command", args: ["cat todos-master.md"]}}
8
```

- also I suspect we'll need something for injecting the agent comms capabilities based on actually supported deeplinks/widgets, eg

```

1 agent_prompt: |-
2   # comms agent system prompt, blah-blah
3
4   in the body section of the response, you can use the following templates:
5   # this will be collected from widgets/endpoints that are installed under the
   widget
6   ${chap_context.response_widget_templates}
7   # or maybe that's also a tool we expose and prerun
8   ${run_tool: {name: get_chap_response_widgets, args: []}}

```

- in general we get rid custom code to maintain, and make it more standards based (a step towards speeding up and scaling up agent development)

↑ Daily Notes - 2025 May

May 23., Fri



Chaps Team

Peter Sipos · in the later morning & around lunch we had a few rounds of testing-fixing with BalintB with the new si...

↑ May 23., Fri

Chaps Team



Peter Sipos

in the later morning & around lunch we had a few rounds of testing-fixing with BalintB with the new simplified broadcast chat-api, which then started to work to send/receive messages (even to newly created threads)

then theme of the day was **to integrate thread histories into the agent client and the whole execution**, this now works, so here 📌 we have an ongoing agent conversation now on the new infra. 🎉

In the second user message I ask what was the first user message, and the agent correctly answers.

```

1 {
2   "messages": [
3     {
4       "id": "msg_32a643d9-09f4-4e2c-b516-ed8df1c89059",
5       "threadId": "thread_a84858cf-88fd-4355-ac2c-6dec9a3f585f",
6       "senderId": "1",
7       "senderType": "user",
8       "content": "Guten tag",
9     }

```

```

10         "type": "text",
11         "metadata": null,
12         "createdAt": "2025-05-22T13:39:14.598Z",
13         "updatedAt": "2025-05-22T13:39:14.598Z",
14         "seq": 0
15     },
16     {
17         "id": "msg_24a898b2-a19d-42e6-b522-19d2344eb6cf",
18         "threadId": "thread_a84858cf-88fd-4355-ac2c-6dec9a3f585f",
19         "senderId": "7b38e6c9-52ea-4654-9ead-b0037f605020",
20         "senderType": "chap",
21         "content": "## <Icon id=\"message-circle\" /> Begrüßung\nGuten Tag! Wie
kann ich Ihnen heute bei der Verwaltung Ihrer Aufgaben in Todoist helfen? Sie
können mir zum Beispiel sagen, ob Sie eine neue Aufgabe erstellen, eine bestehende
Aufgabe suchen oder etwas an Ihren Aufgaben ändern möchten.\n\n<button>Open Todoist
Chap</button>\n> Wenn Sie möchten, kann ich Ihnen auch dabei helfen...\n>
<button>Neue Aufgabe erstellen</button><button>Aufgaben anzeigen</
button><button>Erinnerung setzen</button>",
22         "type": "text",
23         "metadata": null,
24         "createdAt": "2025-05-22T13:39:22.994Z",
25         "updatedAt": "2025-05-22T13:39:22.994Z",
26         "seq": 1
27     },
28     {
29         "id": "msg_e16a9f31-bd45-4f1c-a779-99b49d0825db",
30         "threadId": "thread_a84858cf-88fd-4355-ac2c-6dec9a3f585f",
31         "senderId": "1",
32         "senderType": "user",
33         "content": "what was the first message I sent?",
34         "type": "text",
35         "metadata": null,
36         "createdAt": "2025-05-22T13:40:10.425Z",
37         "updatedAt": "2025-05-22T13:40:10.425Z",
38         "seq": 2
39     },
40     {
41         "id": "msg_d7492953-9269-41eb-a941-4dd9b233211a",
42         "threadId": "thread_a84858cf-88fd-4355-ac2c-6dec9a3f585f",
43         "senderId": "7b38e6c9-52ea-4654-9ead-b0037f605020",
44         "senderType": "chap",
45         "content": "## <Icon id=\"message-square\" /> Your First Message\n-
Your first message was: \"Guten tag\"\n\n> If you want to I can also help you
with...\n> <button>View recent tasks</button><button>Add a new task</
button><button>See today's agenda</button>",
46         "type": "text",
47         "metadata": null,
48         "createdAt": "2025-05-22T13:40:16.368Z",
49         "updatedAt": "2025-05-22T13:40:16.368Z",
50         "seq": 3
51     }
52 ],
53

```

```
54     "hasMore": false,  
55     "nextCursor": null  
56 }
```

Technically this means the agent client fetches thread histories on startup and later maintains them (per thread) as messages come on the realtime channel. The history had to be wired through as far as to orchestra itself so that it becomes input to the agent execution (as we were only passing in the user message before).

Next

in progress Next: deploy this latest chaplet to fly.io, so it can be used used for testing and iteration without running it locally

Then next potential step I have in mind is emitting agent status messages during execution (for which I already have some initial code). I expect this to be a hot topic later as executions take time, and we should be informative / not boring while the chap is working, so it will be a separate world on its own (text and rich status, different style answers etc), similar to final response communication format. But want to start in small steps, so first just relay tool call summaries during execution and start adding control over it (prompting support).

📅 2025. May 23., Fri

ooo as Friday. I will try to arrange for next Friday though, and make it a workday, to push forward the project.

Rebuilt Realtime with an oversimplified but working setup for integration, deployed to Fly.io. Code <https://github.com/lukilabs/chaps-api>. Deployed on push to main.

API and Realtime now sit on the same server, `https://platform.chaps.app`.

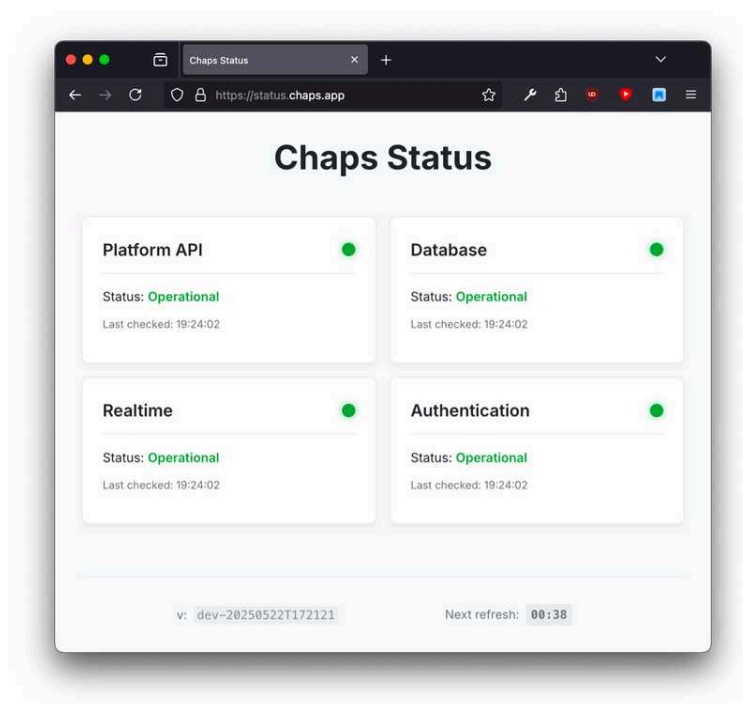
I worked with Sipi today to integrate everything on the API/RT and Chaplet. With this approach, Sipi could integrate the Agents into the conversation flow. See his note above.

Service Status

Built a status service for Chaps, deployed at <https://status.chaps.app> to Vercel. Code <https://github.com/lukilabs/chaps-status>. This checks for all critical components in our stack:

- **API:** implemented health endpoint + check on the API via HTTPS
- **Realtime:** implemented socket connection health check to Realtime
- **Auth:** implemented health endpoint + check via HTTPS
- **Database:** uses Neon driver for a simple health check on our Postgres DB. This is currently on the `db-status` branch, if I can't get it working we'll assume it's up (if API is up)

All checks are real (minus DB for now) and check for the protocol and infra to be running.



Config

Set up static config storage (Cloudflare R2 - S3 compatible object storage - on the Craft main account) served from Cloudflare CDN. Bucket is [chaps-config](#). App (just an example) config JSON can be reached at

<https://config.chaps.app/app.json>

Versioned in <https://github.com/lukilabs/chaps-api/tree/main/clientconfig>. Will write GH action to push to bucket when changed.

Tomorrow

- I'll be spending time writing up what areas Levy should be focusing on and what we are missing for productionising post-beta
- Status update endpoint for Chaps. Status requests in new simple API/RT
- Simple addressbook functionality so we can start chats with users
 - List users/chaps endpoint → for debug
 - Find user by phone number
- Chap auth via Auth service. I didn't have time for this yet, but we'll need a proper way to authenticate Chaps connecting to the API. Currently Chaps use the debug `_uid` param

To discuss

Rate limiting is [dead simple](#) with Express, plug and play, with Redis as a fast in-mem backend. This only adds ~20 μ s per request. What rate limits do we want? I am thinking RLs should be put on creation endpoints (Thread, Message creation) **for the beta**. Specifically: `number of requests / time`, cooling off period when RL hit. RLs keys per user or per connection (per endpoint)? I'm thinking per user - we could get flooded if it's per connection.

Proof-of-concept PR:



Add rate limiting

#1 · Opened by balintb

Open

↑ Daily Notes - 2025 May

May 22., Thu



Chaps Team

Peter Sipos · focused integration work all day · Current status: in theory it should work now to have the first messag...

↑ May 22., Thu

Chaps Team

focused integration work all day

Current status: in theory it should work now to have the first message roundtrip, but there's an issue in realtime, where clients do not get subscribed to new threads that are created. We discussed with BalintB at end of the day, that it should be somehow brute-forced for now (eg running complete resubscription as if client connected just now, when a thread is created), so we are unblocked, and then can fix it up later.

I'm testing now the following flow:

- start chap-platform (locally in docker) → chat api and chat realtime starts listening
- start chaplet (locally in docker)
- create a new chap → agent client connects to realtime as the chap's client
- start client-cli in chaplet-platform, as a debug "sniff" tool to see what's coming on realtime
- create a new thread with Postman with the chap and the user as participant (same user as is sniffing with cli-client)
- send message on new thread from user with Postman
- → client-cli should receive the message (does not get it yet)
- → chap should receive the message (does not get it yet) and ack it back with a new message (POST to chat API)

some topics that came up / took time

- agent client now maintains a proper per-chap connection to realtime, this means it was integrated into the inter-process communication within the chaplet after all, and is getting chap lifecycle events (reacts on chap create/delete)
- had some issues with connection which in the end were because of two root issues:
 - my process `env` was not properly injected, because in our `pm2` process definition the services are not actually launched with their logical working directory, but with `/app`, so that's where `.env` was looked for. I fixed it now for the agentclient, but I think this is a general glitch for all other services, will discuss with BalintB tomorrow
 - and also I was using wrong local address as from within the container localhost is the container itself, but fortunately I found a `host.docker.internal` keyword that points to the machine host (as the chaplet platform runs in a different container, and I did want to compose them all together)
- also had weird slow `socket.io connect()` behavior between agentclient and gateway where it took 20secs to get connected, whereas all other services connected quick once gateway starts listening (this is when chaplet starts). This hindered testing as eg when I quickly created a chap after a restart it was not yet communicated to agentclient, meaning no realtime subscription etc.
- debugging realtime component behavior with BalintB

Today was probably the most frustrating day so far, lots of things not working as expected and previously tested.

With Sipi we were debugging a lot of integration with the Realtime service.

`thread:created` events were not sent properly to all participants of the new thread. This was due to incorrect handling of the participant IDs. These events are now being emitted to the clients connected via Socket.io.

After redoing most of the thread creation logic around API → Realtime comms, messages are now sent on the new thread as well (which, was not subscribing previously) - messages are now delivered multiple times. There's something still wrong with socket handling. Clients can dedupe with the message IDs for now.

The plan is now to ditch thread subscriptions for clients, socket IO will only handle user rooms → this way messages will only be sent to the users room, not the thread room. Messages have a threadID on them, which can be used to route messages on the client side.

This simplification of routing will lead to more messages being sent on the broker, but simplify message routing on Realtime.

This is the highest prio so clients can join a thread and chat with each other (chat in the thread), which - without dedupe on client side - blocks "pure" message handling, one user/agent socketio message to the clients.

As soon as this is resolved I'll work on user/chap handling in Auth as we discussed with Balint in the morning.

Comments From  Balint Orosz →  Balint Bartfai  Peter Sipos

Not sure how noob I am, but I wanted to start running the socket server locally (and check if I can repro the multi message send issue), and I had this issue where I need an API URL, a Broker Setup etc.

I really think a simple dev mode where dependencies are mocked could be super helpful. I.e. there is no possible way to start the socket server without a Redis - and correct me if I'm wrong but Redis (or any broker) is only needed for multi-server (future scalability), use-cases.

Similarly I don't think we need to run the API just to test the socket - a simple file could "mimic" it as a "mock api".

Again, I am no expert in these systems, but :

- Looking at the code, it's simple, easy to read even for me, who is no expert at all in Node
- Looking at the daily notes there is a massive struggle to debug this whole system already (which should be quite easy with a few `print()` statements at this level)
- This usually happens where there are many components and no easy way to monitor/modify each in parallel.

→ My logic is to simplify the remote systems to "static" local ones, so we can isolate the issue itself easily

(I see there is a `ALLOW_DEV_THREAD_IDS` and `USE_DEV_AUTH`, so maybe we are already 98% there, the app was just crashing when Broker and JWT wasn't provided)

I do this all the time when I don't have a backend / want to simulate a backend capability not available yet.

↑ Daily Notes - 2025 May

May 21., Wed



Chaps Team

Peter Sipos · most mostly collaboration day: company meeting, web guild, chaps integration discussions with Balint...

↑ May 21., Wed

Chaps Team



Peter Sipos

most mostly collaboration day: company meeting, web guild, chaps integration discussions with BalintB

Some proceedings:

1. **started agent integration to the chat platform**, totally in-progress, this will be tomorrow's focus. I added a new service in the chaplet, which will be responsible for this communication, and acts as internal (end) client towards the existing system. It will maintain request/thread state etc , and implements a message transformation layer, between our legacy messages and the new APIs.
There are only a few tricky parts, one is **chap auth**, how to provision the auth token to the chaplet (or chaps). I think it would be nice if the user and agent networking could go with the same "middleware" (that manages the token) but if not, we will survive that as well.
2. had detailed **discussion** with BalintB about the **status layer** (ephemeral participant states, like "typing" and "thinking") in the realtime component. We agreed realtime will provide a functionality to send all participants status on connecting, will either need a manual API trigger call, or automatic on connect. From then it's regular pub/sub messaging.
3. also **played more with todoist MCP chap**, some learnings:

sidenote: in first rounds of searches the most popular hit, and the most forked repo was just a garage project MCP, I had to look for a more serious todoist MCP, and it turns out there's actually two quite complete ones (first is the official, but cannot be installed simply, so went with second)



Doist/todoist-mcp



stanislavlysenko0912 / todoist-mcp-server

Now these expose a much bigger api surface: `Successfully connected to MCP server todoistApi` with **34 tools**

Checked if this is feasible from token usage perspective, and we're good: eg the above 34 tool descriptions are quite condensed, and the whole tool defining user prompt is just ~1.4k tokens, and whole processing was in the 6-10k range. (earlier when we had super inefficient 10-15 LLM rounds we had even 50-100k token consumption, of course that's not a very good baseline)

There's an interesting aspect though, because of the concise and sometimes incomplete tool descriptions, the LLM (4.1) **sometimes makes invalid requests, but so far always recovered afterwards** with repeated requests.

for example todoist `update_tasks` call requires `content` prop, but tool description does not mention it:

```
1 - update_tasks: Update tasks in Todoist
2 Either 'task_id' or the 'task_name' to identify the target.
3 Parameters:
4     items (required) (array):
```

so LLM first makes invalid request, but the api tells the issue, so it is easy to recover

Lots of discussions around integration and architecture with Sipi today. We got platform working for him locally, he's started on the Agent side of integrations.

We came up with a way to relay status information to the client, which is not persisted in the DB. Status messages are written to a cache (in this case, Redis), and passed to the client on request.

This has been implemented on the API and realtime side - however, no clients are passing status info yet.

Platform

API has been deployed to Fly.io today. Currently it's reachable on `chaps-api.fly.dev` and `platform.chaps.app` (will be `api.chaps.app`).

Realtime is running on `realtime.chaps.app`.

Auth has been integrated on the User side to platform. Clients connecting to platform API are authenticated with the same session tokens used to call previous API endpoints (login → auth token → session token). The same session token is used by platform API to connect to and call Realtime (for message sending), and the same token to connect the (app) client to Realtime.

API Postman docs are ready here: <https://luki-labs.postman.co/workspace/Chaps~8eddc1e8-9c93-4ba9-aaa6-83f26c89c186/collection/26269984-d8b604a2-5fed-453c-8d98-e8a479c335c7?action=share&creator=26269984&active-environment=26270218-412b04a0-baf0-455a-a806-bd984849e24a>

Wrote docs - see [Connecting clients to Platform](#) for endpoints and instructions.

Please be mindful when integrating that the message structures might differ slightly, I will update the docs if there is a discrepancy - or change them if need be - full E2E testing with message sending and receiving has only been done with our own client cli and monitoring redis. I have changed some internals, have not had time yet to go through every single one to verify.

Next steps

- Chaps authentication. We have auth, but we do not have a way yet for a chap to identify itself to the APIs - login flows are not designed for M2M scenarios, only TOTP for users. We can use the debug `_uid` params for testing until this is solved. I'll figure out something tomorrow.
- Thread update (participants joining, leaving) API and messages on RT. This can't be done right now via API.
- Work with Sipi on the Chap client so that they can actually chat on the same thread.
- ...and I'm anticipating some issues cropping up when we start fully integrating, those will need to be fixed.

Comments from  **Balint Orosz**

- We will likely need an "Address Book" api or similar (for both storing contacts – people and chap?)

↑ Daily Notes – 2025 May

May 20., Tue



Chaps Team

Peter Sipos · started drafting the API messages 📌 · Chap message API (after pub/sub chat server setup) · In the m...

↑ May 20., Tue

Chaps Team

 Peter Sipos

started drafting the API messages 📌



Chap message API (after pub/sub chat server setup)

Message API proposal • Messaging use cases • Table • type == "message" • This is the main user...

 Last Edited 5/19/2025

In the morning I did reach a stable state with todo-list widget and commit experiments which I merged. (though we agreed on workshop, we may not need LLM commit for API edits at first, I wanted to bring the branch home, will clean up later as needed)

I also span up a [todoist MCP](#) Chap, which I'll use to experiment with communicator response and other dynamics of MCP chaps. It should have worked in a few mins, but I did lose some time on some silly issue in our own Python code, which was cumbersome to debug as it needed always rebuild/recreate chap cycle. Anyway, now ready to progress on MCP front.

Spent most of the day fixing socket (+event) handling in Realtime API. Clients connect to RT via Socket.io. When a client (the iOS app) connects, Realtime subscribes to broker messages with specific topics - these are `thread:{threadID}:*`, `user:{userID}:*`. When a message arrives from the event bus (Redis), Realtime sends these events to connected clients. Clients are segregated (by user ID), and different message types are sent via Socket.IO rooms, eg. `thread:{threadID}`, to the client who is monitoring those events. There were multiple issues I had to fix

- Clients were on a different namespace (`/user`), which resulted in messages not being passed along
- In some cases sockets were being used before the handshake in Socket.IO - resulting in the same as above
- IDs were being generated with `:` in them - topics were not parsed correctly
- Redis needed `pSubscribe()` instead of `subscribe()` for topic subscriptions

(and some more issues along the way)

I've deployed Realtime to Fly.io - currently this is running at `realtime.chaps.app`.

I've been working on deploying API to Vercel, but I haven't got it working yet. If I can't solve this tomorrow morning, we'll run API on Fly.io as well.

Tomorrow I'm working on these to be ready to start client integration Wednesday:

- API deploy to Vercel / Fly.io
- full docs for API, Realtime
- Auth on API. JWT handling

I'm not anticipating the (App) client-facing interface to change. We don't have working Agent clients connected yet, client integration should start with the chat functionality, which we can plug in Agents a bit later.

What **won't be done by Wednesday** but is needed to integrate all components:

- Agent client, connecting to Realtime
- Chap creation, deletion via API → RT → Chaplet
- Agent messages on Chaplet
 - Rewrite gateway on Chaplet to handle Realtime messages (client → Chap, and responses from Chap → client)

We have a working API and Realtime layer, Agent integration can start, let's divide the work

 2025. May 20., Tue .



Chaps

Balint Bartfai · Today I built tooling (cli app) to test the API layer (see [repo](#)). I've been running E2E testing, and using...

↑ May 19., Mon

Chaps

 Balint Bartfai

Today I built tooling (cli app) to test the API layer (see [repo](#)). I've been running E2E testing, and using that tool, I've identified a number of issues with the current setup around message and socket handling, which I'll need to work on further:

- client can send a request to API with the session token nearing expiry → API connects to Realtime but is not authenticated. I'll need to either regenerate session token on API layer or trust clients to refresh tokens on time. Generating a connection token on API would make sense, which could contain the thread IDs as well, saving the roundtrip from Realtime to API for initial subscriptions
- if a connection with an invalid session token is created from API to Realtime, due to the way sockets are handled, that connection is persisted, even with an invalid session token. I'll need to redo socket handling to close these connections as soon as the auth fails and not wait for the idle timeout

Chaps (Wed-Fri)

Implemented most of Platform <https://github.com/lukilabs/chaps-platform>. API and [Realtime](#) are running (local Docker), infra components to deploy set up.

Next steps:

📌 I will be **moving Auth codebase into Platform** for us to be able to run all in one docker compose spec.

- Realtime Agent endpoint
 - Right now, "human" clients can connect, their topic subscriptions are handled. Agents need different subscriptions to different topics. Connection and auth logic is almost the same.
 - Library to connect as an Agent, which will be running on the Chaplet
- Productionise platform:
 - Deploying RT to Fly.io and API to Vercel
 - Hooking up (production) Auth

Executive summary

We have two new exciting areas in focus regarding Chap internal logic:

- hybrid chaps that have both a traditional API/web interface and an LLM powered chat interface.
- Chaps that work not (just) on local data, but connect to external data and capabilities via MCP connectors.

Last week had promising start and progress on both fronts (see vids for progress in last week dailies). I see a lot of future work on several different areas, but quite confident in feasibility.

The most challenging technically will likely be runtime isolation for widgets, which we may not need initially, only if we want to support 3rd party code chaps.

The most challenging/ambiguous from product perspective seems to be the logic that's responsible for the actual user response / communications output (format, widgets content, action buttons, different response based on different use cases etc)

Detailed writeup of hybrid Chap architecture



Hybrid Chaps

The new, hybrid, empowered chaps roughly look like this · Image · basically we aim to combin...

Last Edited 5/21/2025

Potential work tracks for the week

in-progress (~half day) Finish an initial version of the API edit commit lifecycle, the goal here

- run commit lifecycle concept with BalintO (related [docs here](#))
- have an architecture and message flow that supports imagined widget use cases
- have a successful e2e demo where LLM smartly updates consequences of API edits, eg user checks a todo on UI, LLM removes related reminder and summarizes changes.
- the actual communication (message publish) generated from API edits will be dummy/highly PoC (this has to be explored from product side, and we may not even always need it)

What's needed for a todo Chap?

Lot of feature development but that **needs product input / actual iteration** for example

- format of response messages
- showing specific todos in a response widget? (not full list, but ones affected)
- full screen todo list widget with dnd reorder support?
- full screen date picker if user taps on a due date in agent response?
- resume evals
- image support?

can be started if we know that we want manually sortable todos then revamp storage logic of todos in todo chap to have a master file with active ordered todos

- will need to change LLM agent prompts
- manage linked files with details
- evolve todo-list widget to support (demonstrate) DnD reorder

- consider archiving from master file
- etc

Messaging platform integration track depends on Realtime status maybe second half of week?

- integrate the Chaps into new Realtime messaging
- revamp in-progress messages → figure out new "status" messages
- figure out notifications (email/sms)

If we want to release with MCP Chap(s) can be started

- figure out communicator response for MCP chaps (likely based on MCP call history, or maybe chap specific endpoint calls to diff)
- proper configuration widget (these will need API key etc configs that user needs to enter on a UI), likely a general, themeable one that can just be mixed into any Chap
- work on Chaps build pipeline → not store them in chaplet/shared/templates, but have their separate place where we produce a zip that can be installed

↑ Daily Notes - 2025 May

May 16., Fri

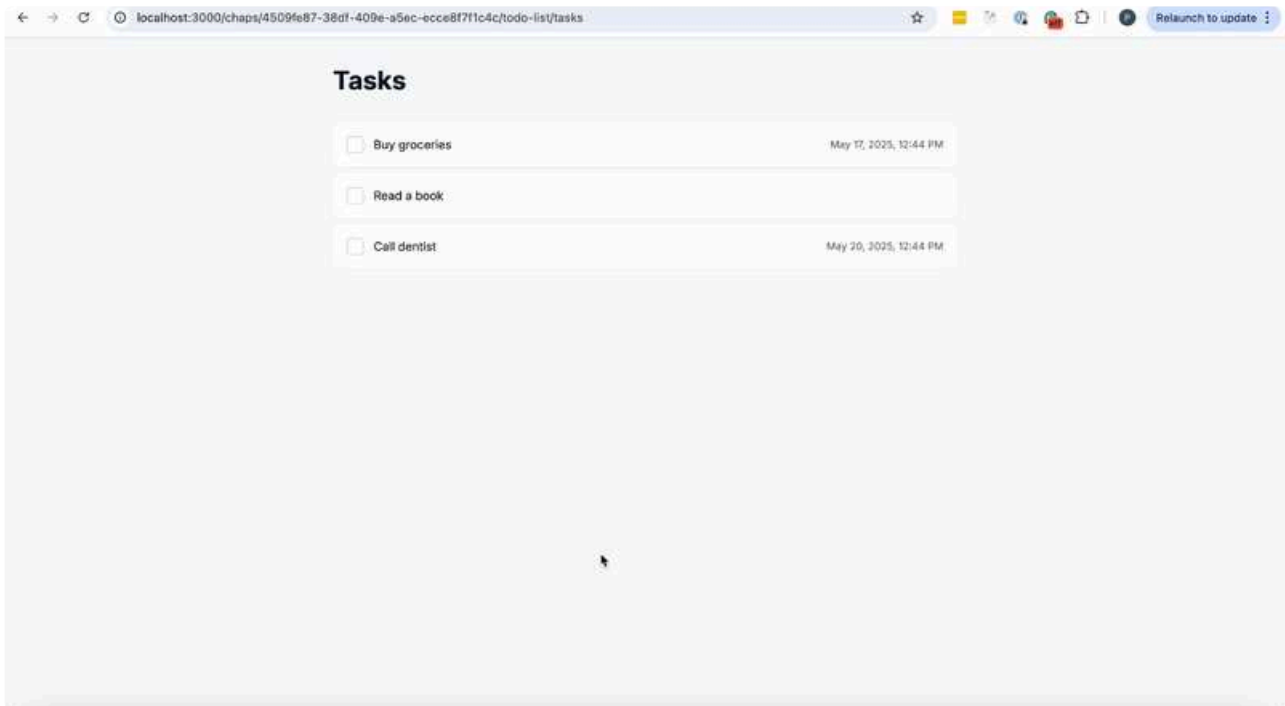


Chaps Team

Peter Sipos · Video · The app, underlying API etc was fairly straightforward to do, however I did have to plumb and d...

↑ May 16., Fri

Chaps Team



The app, underlying API etc was fairly straightforward to do, however I did have to plumb and debug on the route integration and supporting formdata in the widget etc so some lower level tech stuff, not super important in grand scheme of things, though important learnings for later widget development.

I also started working some on the commit logic after user edits, but that's still WiP. With that we'll have an e2e widget.

There has been a lot happening under the hood, I will do a more proper writeup before the weekend (into Monday's daily notes), about what I see as challenges, tasks etc.

 2025. May 16., Fri I'm calendar-wise ooo, but will catch up with documentation for next week

Chaps [Platform](#)



lukilabs / chaps-platform
Platform for Chaps



lukilabs

TypeScript

Stars 0

Finished working API implementation. Implemented a minimal Realtime service. The service receives socket connections from clients and subscribes to topics on the Redis broker which are "relevant". Relevant topics are messages for the user, the threads that they are in, and high-signal system events.

All connections (user → API, user → Realtime, API → Realtime, Realtime → API) are authenticated with the user's session token, locking down the request(s) to that user.

On connection to Realtime, Platform API is called for a list of threads the user is subscribed to. This forms the starting set of subscriptions. As new threads are created / threads are removed, these topic subscriptions evolve inside Realtime.

API is used to send messages, which sends them in to Realtime. API maintains a connection per user, which is not closed on request end, unless an explicit `X-Close-Realtime` header is sent. Connections have a max idle time and are closed after that. This is to not incur the overhead of connection & handshake on every message send request.

Both API and Realtime are capable of running on the edge (we do not plan to run Realtime on the edge, yet). Persistence is separated from both services. Realtime maintains persistent client connections via Socket.IO - however, when the Socket.IO server disconnects, the client will retry and most likely get routed to a fresh running instance.

Both API and Realtime support a dev-only authentication mode: setting the user ID via query params, and for Realtime, setting the thread IDs watched via param. This allows each service to run without the other for dev.

Every service supports running locally via Docker. Docker compose can run the whole platform with all dependencies (API, Realtime, Postgres, Redis) in one go.

Built tooling to:

- monitor Redis event bus directly
- connect to Realtime as a user client via socketIO, monitor messages

Tomorrow:

- Continue Realtime. Testing and fixing issues found while E2E messaging.
- Linking with Auth service.
 - Possibly moving Auth API into the Platform repo
 - Proper testing with JWT tokens. I have been running with dev auth so far.
- If I still have time, will start to work on deploying these outside of localhost



Chaps

What's going on in the world of AI? 🧠🤖 · ChatGPT can now use Github repos for Deep Research. This brings Chat...

↑ May 15., Thu

Chaps

What's going on in the world of AI? 🧠🤖

- ChatGPT can now use [Github repos for Deep Research](#). This brings ChatGPT closer to the more "traditional" coding tools - Windsurf, Cursor, Claude. Yes, it already has "work with apps", but in my experience they tend to be focused on single-window context.
- Mastercard is pioneering agentic payments. It's [Agentic Payments Program](#) seems to be a mix of an MCP and an Agent (A2A? it's not clear from their press release) - however, it's not the wild west MCP anymore, they require vetting of agents to be able to connect. In my opinion this marks the start of a very gradual (financial world...) adoption of agentic systems for payments and commerce. Mastercard seemed to have moved fast here.
- Perplexity's [valuation is now at \\$14B](#), with a new funding round of \$500M.
- Notion has jumped on the AI bandwagon with an [agentic research assistant](#). By the looks of things they have essentially just rounded up a bunch of their integrations into MCP servers and delivered an agent on top.
- A very interesting model concept has surfaced, the [Continuous Thought Machine \(CTM\)](#). "Traditional" neural networks, the OG AI tech (Transformers, CNNs, LSTMs, etc.) treat every neuron like a light bulb that's either on or off to some degree *in a single instant*. They ignore *when* different neurons fire relative to one another, even though real brains care a lot about timing. CTMs give access to a neurons brief history, "capturing" its own "thought process". These models claim that, given longer thinking times, are better at reasoning than just "single-state" generative models. There's a lot of drawbacks right now (costly to train, it's really just a research prototype), but the concept is very forward-looking!
- Interesting article on [conversational interfaces](#), how UI changes in the world of chat bots. *Spoiler alert: Chaps is also aiming to revolutionize this area.*
- TikTok released a new product called "[AI Alive](#)", which generates videos for your stories from images. Remember when Facebook had spatial/3D effects on photos? This type of innovation is the next step with todays technology. No, I don't have a TikTok account yet, but yes, I will try it out.
- Google's Gemini assistant is being pushed to [more Android devices](#), like TVs, watches, and Android Auto. They're moving much faster than Apple in this area...

 Balint Bartfai

PSR.

Chaps Platform

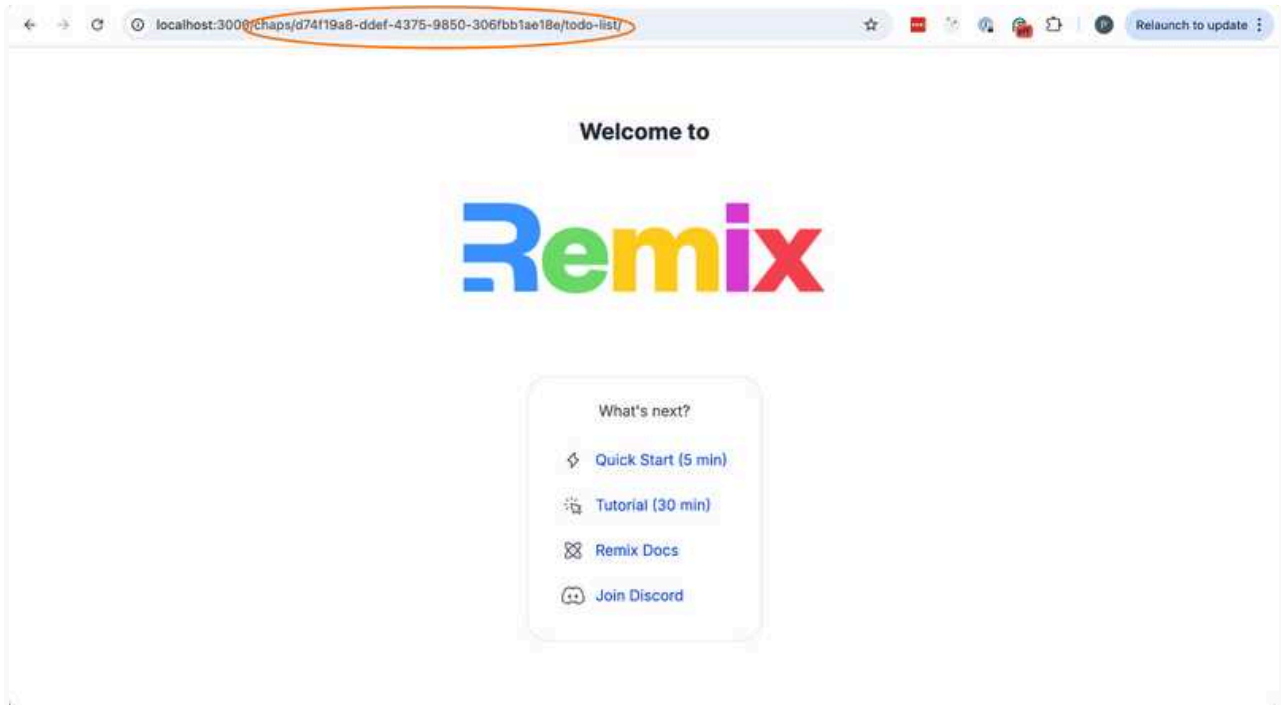
Heads-down work on the Chaps API (renamed from Messaging API) component of Platform. See [platform](#) for more info on how this fits into the big picture.

Setting up persistence layer and infra in <https://neon.tech/> (Postgres 17).

 Peter Sipos

Also did PSRs in the afternoon

Will have some more detailed writeup later, but on Chaps front I was plumbing support for chap widgets. I reached this stage



doesn't look much but it's actually quite tricky, this is a small web app in Remix, that is built standalone, and then mounted under the *dynamic route of the chap instance*. Meaning an agent dev can (soon) build web apps, which can then run as chap widgets. The complexity was in the adapter layer, for which Remix has some support (to be mounted in an express server along with other web apps), but in the end it turned out it won't work with a dynamic base path, only static one. People were using reverse proxies to make it work with a dynamic path. Ended up solving anyway 🐱

Why Remix? It is fairly lightweight (not super-lightweight if using React, but fairly, it's to be tested, maybe can strip it down to Preact later), and has great support for server-side rendering which I think we really need for widgets, for two reasons:

- we want them to load *fast*, we don't want a client to fetch the web app, then the web app fetch the data with two network trips to see a widget in a message thread. With SSR we do the fetch on server side so it will be one request.
- in the workshop we discussed with Balint (O) that we may want to use captured widget images in the conversation flow. I'm very much interested in having inline interactive widgets, but performance or UX reasons may call for using images. SSR is a great friend here as it should make capturing the widget pages to images quite seamless.



Chaps Team

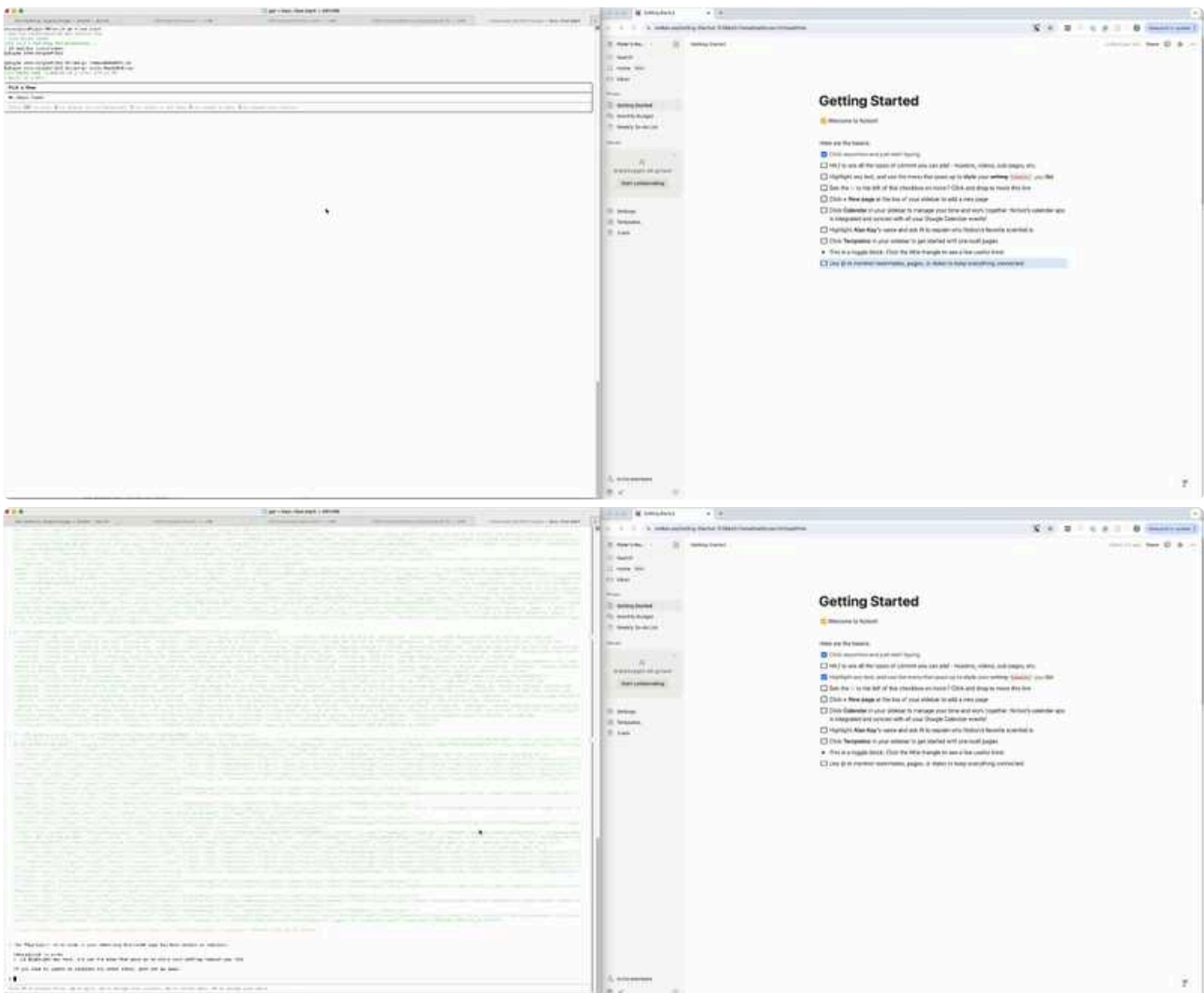
had follow-up architecture workshop on Tue afternoon · Peter Sipos · implemented an PoC Notion Assistant Chap vi...

↑ May 14., Wed

Chaps Team

had follow-up architecture workshop on Tue afternoon

implemented an PoC Notion Assistant Chap via MCP (this is not a standalone demo, but runs with our existing agent system, running the official [Notion MCP Server](#) locally)



I added a new MCP tool config option to the agent's config.json file:

```
1  "tools": [  
2      {  
3          "type": "mcp",  
4          "name": "notionApi",  
5          "command": "npx",  
6          "args": ["-y", "@notionhq/notion-mcp-server"],  
7          "env": {  
8              "OPENAPI_MCP_HEADERS": "{\"Authorization\": \"Bearer  
9              {{instanceConfig.notionApiKey}}\", \"Notion-Version\": \"2022-06-28\" }"  
10         }  
11     ]
```

The other piece is that I started building out the Chap web endpoint capability, and now Chaps can define their own routings which can point to api endpoints or complete web apps within `/meta`.

Note: I do not put the handlers in an execution sandbox, so this initial version won't be good enough for supporting 3rd party chaps (handlers in one chap should not be able to access the whole chaplet file system, which includes other chaps, and should not be able to bring down the common webserver etc).

Also I would like to iterate on the actual solution a bit, so that it's easy for chaps devs to produce a conform server handler output, which is picked up by the chaps webserver (currently: gateway itself)

I leveraged this to add a configuration endpoint to the Notion MCP chap

```
1 // in /shared/src/templates/chaps/notion-mcp/meta/routes/appconfig.ts
2 export function configureRoutes(router: Router, chapId: string, metaDir: string):
  void {
3   // POST /api/configure - Save configuration key-value pairs
4   router.post('/api/configure', function(req, res) {
5     ...
```

which in turn saves config to an `/meta/instance-config.json` file, and this config can be used as placeholders in the MCP tool config

```
1 ..."{\"Authorization\": \"Bearer {{instanceConfig.notionApiKey}}\"...
```

Next:

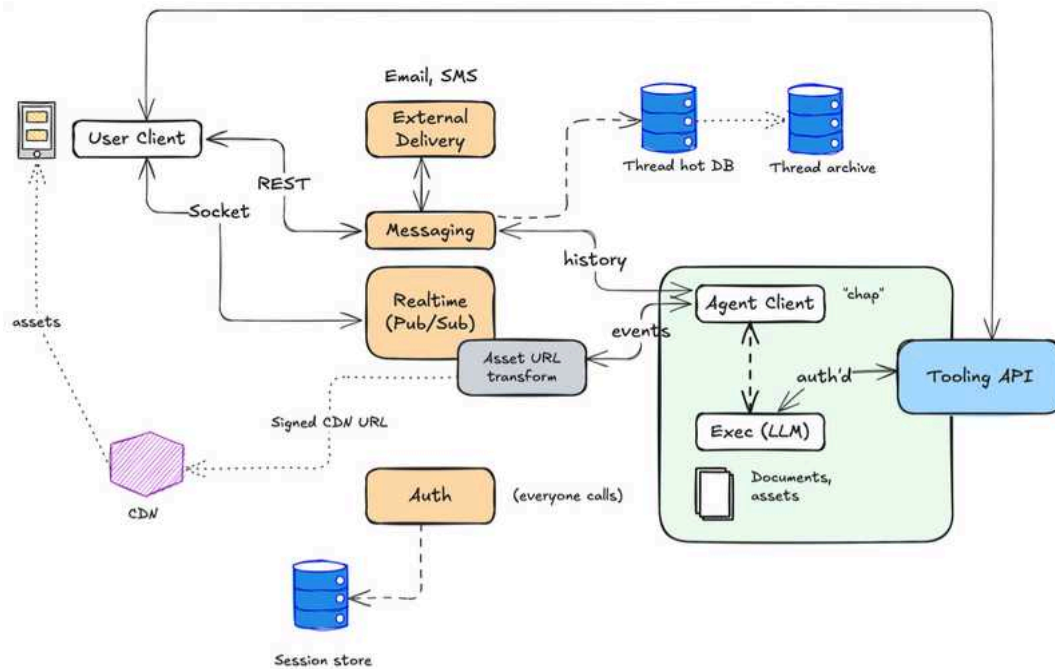
Two main topics to continue for this week:

- start building web app endpoints for the todo chap (listing todos, checking todos, maybe reordering todos etc)
 - including agent processing of direct content updates coming from these endpoints
- explore "communicator layer" in MCP-based chaps, where eg we don't have file diffs (which is an input to communicator agent)

Chaps Platform planning and technology evaluation today

Designed MVP for Chaps Platform. We had a longer catchup with Sipj, Balint around architecture, and are now aligned on that.

▼ Overview diagram





Chaps Platform

Image · Entities · Schemas · Events · Messages · Threads · Services · Auth · Messaging API · ...

 Last Edited 5/20/2025

Tech stack will be Redis for PubSub, Typescript for services, Socket.io for client communication.

Minor infra:

- Moved over Auth from Vercel's Upstash Redis to our own Upstash account.
- Auth is now `auth.chaps.app`

...

Next steps for the week ahead:

- Messages API MVP. This involves
 - Infra setup with Vercel and hosted Postgres
 - Thread and Message basic schemas, DB structure
 - Service in Typescript
- Realtime MVP implementation
 - Fly.io infra: Docker, routing

- Client connection handling with Auth (Human/Agent clients)
- PubSub subscriptions to authorised events (=topics on pubsub)
- Routing messages according to topics
- Docker image and Docker compose for complete Platform:
 - Realtime
 - (Messaging) API
 - Auth

↑ Daily Notes - 2025 May

May 13., Tue



Chaps Team

Peter Sipos · goal for Tue, 13 May: built a demo Notion MPC Chap · Exploring MCP Chaps · Introduction - Model Co...

↑ May 13., Tue

Chaps Team

goal for  2025. May 13., Tue : built a demo Notion MPC Chap

Exploring MCP Chaps



Introduction - Model Context Protocol

Model context protocol (MCP) - OpenAI Agents SDK



Using MCP Servers with OpenRouter



makenotion / notion-mcp-server



mcp_adapter.py

TL;DR: MCP is a way to provide tools/function calls to LLMs.

→ we still need LLM on the caller side, and Orchestra, OpenRouter etc all have now support. Basically it's a very similar situation to Craft chaps, except it has a different set of tools → eg instead of using file tool to manage locally stored data, it uses provided MCP tools to manage stuff, and any data resides remotely, but we still need agent metadata, communicator response formatting etc.

Basically a Notion Chap needs

- a way to configure access token (eg hosts a simple config webapp, which saves it a well known config file in /meta)
- can typically just run MCP server with `npx` have, so don't even need a custom npm package per chap for this (just an MCP config as part of the agent)

Implications:

- we don't have git diff, cannot use diff based communicator agent to generate output (what is the equivalent strategy? - depends on available tools)
 - though in theory can do diff by fetching before and after
- since data is not encapsulated by chap, evals state setup is more complex

Exploring a native Calendar Chap

we could reduce this to an MCP question

people already proto/demo MacOS calendar MCPs (these are demo repos)



Omar-V2 / mcp-ical



shadowfax92 / apple-calendar-mcp

but eg a big one for Google Calendar



takumi0706 / google-calendar-mcp

However since we need to access the calendar in the client app, this would require:

- we need an "MCP calendar module", a piece of iOS code, which exposes calendar functionality as MC tool call like api
 - this is either built-in code in Chaps app, so not extensible by 3rd party devs
 -  or 3rd party devs can build an Chaps-Calendar iOS app, and we need a push server to facilitate comm between running Chaps app and background worker app
inter-app communication flow: · Snippet
- proxy the iOS module's MCP functionality as a local MCP server in the chaplet. We establish a transport channel to the Chap which - from the perspective of the Chap - exposes the "remote" MCP running within (or next to the) Chaps iOS app, as a local MCP server running in the Chap container.
 - the transport can be either part of the chat itself, as low level tools messages exchanged in same pub/sub
 - or maybe Chaps can define a well known "host MCP endpoint" which is a long poll/socket endpoint to Chap app that facilitates MCP calls

Architecture refinements with BalintB

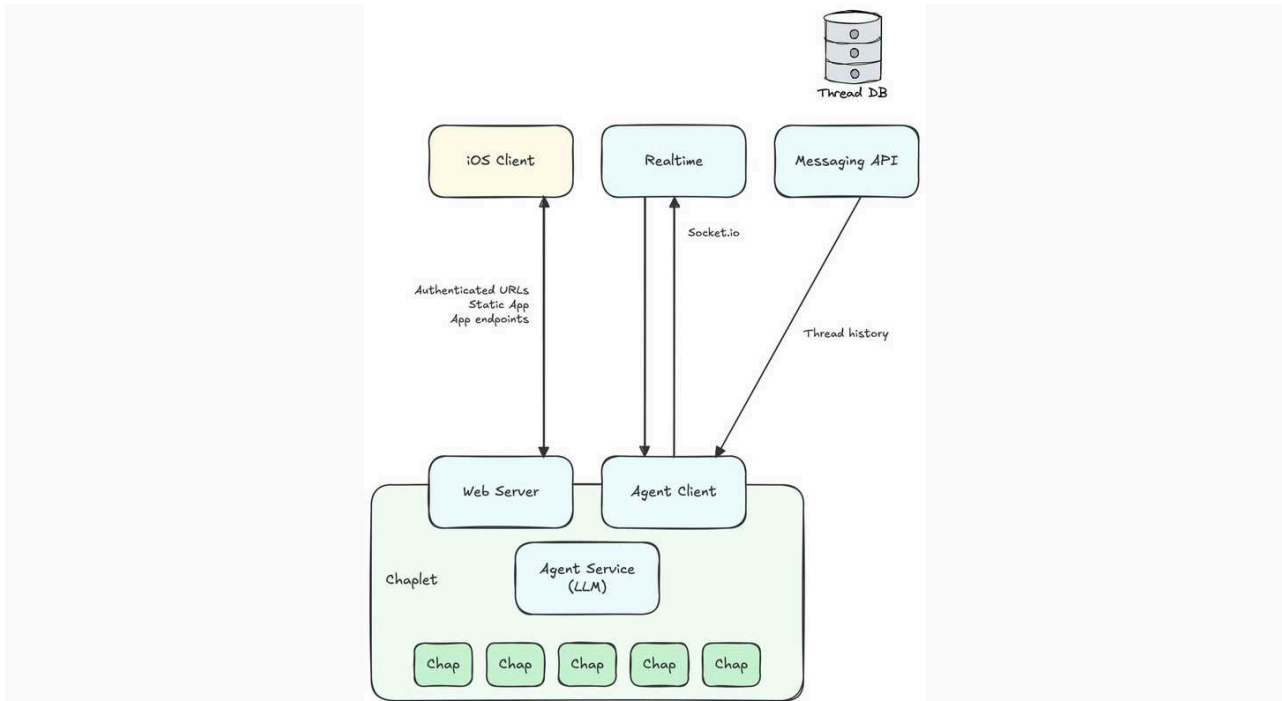
had a long discussion with BalintB about several parts, mostly the agent client, the new web endpoints of chaps. Some highlights:

- at first the *agent client* can be part of the agent service (former orchesra service), this is responsible for
 - managing the pub/sub connection
 - retrieving history from chat server
 - when a new prompt arrives, it assembles the history context for the agent invocation (this involves eg mapping to internal LLM messages from user messages, assigning identity to messages in multiplayer conversations etc)
- there will be a new service and component in chaplets, which is a *Web Server*
 - initially each chap can have a *server.ts*, which will list routes the chap handles (and pre-built web apps)
 - web server component handles *server.ts* of all chaps and implements routing so that proper web app is served
- Chap web endpoints
 - we could do a built-in web app for all chaps called *configure*, which takes a json descriptor of user facing chaps configurations, and exposes a UI to fill those props (and saves it to a chaps config file in */meta*)
 - for example a Notion MCP Chap will need a Notion token to be specified by user
 - the web apps (both page load and api calls) will need to be authenticated with session token

- the endpoints will be able to run chap defined code, including making changes to files. Likely we will need a commit call as well, which will invoke a commit-direct changes agent entry point, which runs syntax fixer, summarization etc, git commits and posts to pub/sub.

Agent Client planning with Sipi

We discussed at length and agreed on the high-level architecture for the "agent client".



The agent client is responsible for

- all realtime messaging
- history fetch
- history "enrichment"
 - an agents history consists of "internal" and "external" messages
 - the agent is only interested in its internal history for state - external history is shown to the human client

The agent client is NOT responsible for

- subscribing to the correct topic. This is handled by Realtime, with access control managed by Auth

Web server

An interesting aspect of Chaps are they are mini web apps. A minimal static server is needed to serve these apps. The web server is responsible for

- Authentication: checking token validity
- Routing: deciding on which Chap to route to

Due to the Chap routing it might seem contradictory to place it on the "agent client layer". Chaps are responsible for serving their own content, but our Chaps are run on Chaplets, therefore we need this extra service. For now, we'll keep it in this layer.

Web server - Dynamic content

Debated the possibility of having dynamic routes. Why?

A user interaction with the mini app could have calls to the app's "backend" for persistence. For example, let's take a look at an example. The Todo chap shows a list of todos. The user ticks a checkbox. What happens?

1. The app makes an auth'd call to its backend (dynamically routed via web server) to tick the checkbox. This is not very transparent to any other threads with this chap in it.
2. User interactions generate prompts (flagged as "generated" or similar), which are sent through the client, executed in the chap, and ultimately sent on the thread's topic to subscribers.

Let's discuss 📅 2025. May 12., Mon .

Technology selection

- **Auth:** TS, Redis, JWTs. Todo for production: **RSA signing** instead of symmetric secrets.
- **Messaging:** TS, SQL ORM ([Drizzle](#))
 - Should be able to run on edge. Options:
 - Vercel edge, Postgres
 - Cloudflare workers, D1
- **Realtime:** Message bus options without vendor lock-in:
 - Redis PUBSUB. Simple single-threaded. Hosted solutions available. *Need to test concurrency performance*
 - Apache Pulsar: hosted solution (<https://streamnative.io/deployment#serverless>) not convincing, pricing not transparent. Did not find other hosted solution. Hosting it ourselves is **not an option** due to complexity.
 - Kafka: similar to Pulsar. Higher complexity to set up and run.
 - NATS Jetstream: fixes NATS "fire-and-pray" style messaging. Not many hosted solutions, some built on NATS but vendor-specific platform (eg. <https://www.synadia.com/>) - for 50USD we get 100 connections - for the same price on CloudAMQP we get 1,5k. Self-hosting **not an option** due to complexity.
 - **AMQP protocol**: Hosted options available - eg. <https://www.cloudamqp.com/> -can be durable.
 - Hosted option LavinMQ seems like a great alternative, but *would lock us in* to only one hosted solution, unless we can use standard AMQP
 - RabbitMQ supports standard AMQP
 - We should standardize the broker protocol, not the queue tech
- **Realtime: websockets** on the edge
 - Socket.io running on scalable nodes (fly.io, CF edge, etc).
 - Cloudflare workers can keep socket conns open for hours, or
 - Custom WS on Fly, etc.
 - Keep this on the edge: connections should be distributed between nodes

Realtime: how we stay vendor-agnostic

We wrap the broker implementation behind an adapter in code. Business logic will only ever talk to a simple `pub()` / `sub()` interface. The message format (envelope) will be defined, transport layer can be switched. *CloudEvents for envelope?*

Tomorrow

Tech for Realtime continued. I'm going to put together an extremely simple proto to see what devex is like for pubsub via AMQP: topics, queues, queue limits, durability.

Simple edge SocketIO on CF, Vercel edge (?), connected to AMQP Broker.

Continue architecture by specifying Realtime messages and Messaging API calls.

↑ May 13., Tue · Chaps Team

or 3rd party devs can build an Chaps-Calendar iOS app, and we need a push server to facilitate comm between running Chaps app and background worker app

inter-app communication flow:



27		App A			App B	
28		updates UI			goes idle	
29	+-----+			+-----+		

↑ Daily Notes - 2025 May

May 12., Mon



Chaps

Balint Bartfai · Chaps infra · There have been multiple resets of user IDs over that past few days due to development...

↑ May 12., Mon

Chaps



Balint Bartfai

Chaps infra

There have been multiple resets of user IDs over that past few days due to development of the auth service. We have 9 orphaned (but running) chaplets. Cleared these up.

Removed old DNS settings to sunset login2.chaps.app.

I added a read replica in FRA for Upstash (Redis). Currently it's not necessarily needed for scaling, but will help us with 1) development - reads are much closer and faster, and 2) we can test how and if eventual consistency impacts the auth service. So far, from my minimal stress testing, reads have never been stale even immediately after writes. Auth is read-heavy.

Removed old Redis cloud database.

Chaps Auth

All changes documented in the guide: <https://s.craft.me/hCpSZGCgtzYHH1>

Token and user related Redis keys have been divided into more keyspaces. The keys have changed, you'll need to login again.

- `chaps:token:{tokenHash}` → `chaps:token:hash:{tokenHash}`
- `chaps:token[set, used, geo]:{tokenHash}` → `chaps:token:[set, used, geo]:{tokenHash}`

- `chaps:user:{uid}` → `chaps:user:id:{uid}`
- `chaps:username:{unameHash}` → `chaps:user:name:{unameHash}`

Added endpoint to modify user profile - currently this can only set/unset the users username. User profile CRUD is complete now. `PATCH /api/v1/user`

Added route for `/logout`, keys are now handled / deleted atomically.

I have also added the capability for auth service to list where the user is logged in, and the ability to delete a login (different than the current one):

- `GET /api/v1/token/list`
- `DELETE /api/v1/token/{tokenHash}`

Geo information (City, Country, IP) is saved as well. This enables the client to display a "where am I logged in" list.

As the Auth service grows in complexity, I've started to [document](#) everything in the repo. I've written the [docs for Redis keys](#).

 Peter Sipos

Admin: having a dentist appointment on Thursday morning (hopefully getting to the end of it soon), so I'd request for postponing the Chaps standup to 11:00

[Last Thursday's update](#)

 2025. May 12., Mon workshop with two Balints on chaps communication topics

(weekend) added [my notes / draft plan for tomorrow](#) incorporating the new chat server concept (in the main/orig workshop doc)

↑ Daily Notes - 2025 May

May 9., Fri



Chaps Team

Peter Sipos · ooo today (default Friday) · Migrated evals tool to new auth · can just sign in now with bun run auth (wi...

↑ May 9., Fri

Chaps Team

ooo today (default Friday)

Migrated evals tool to new auth

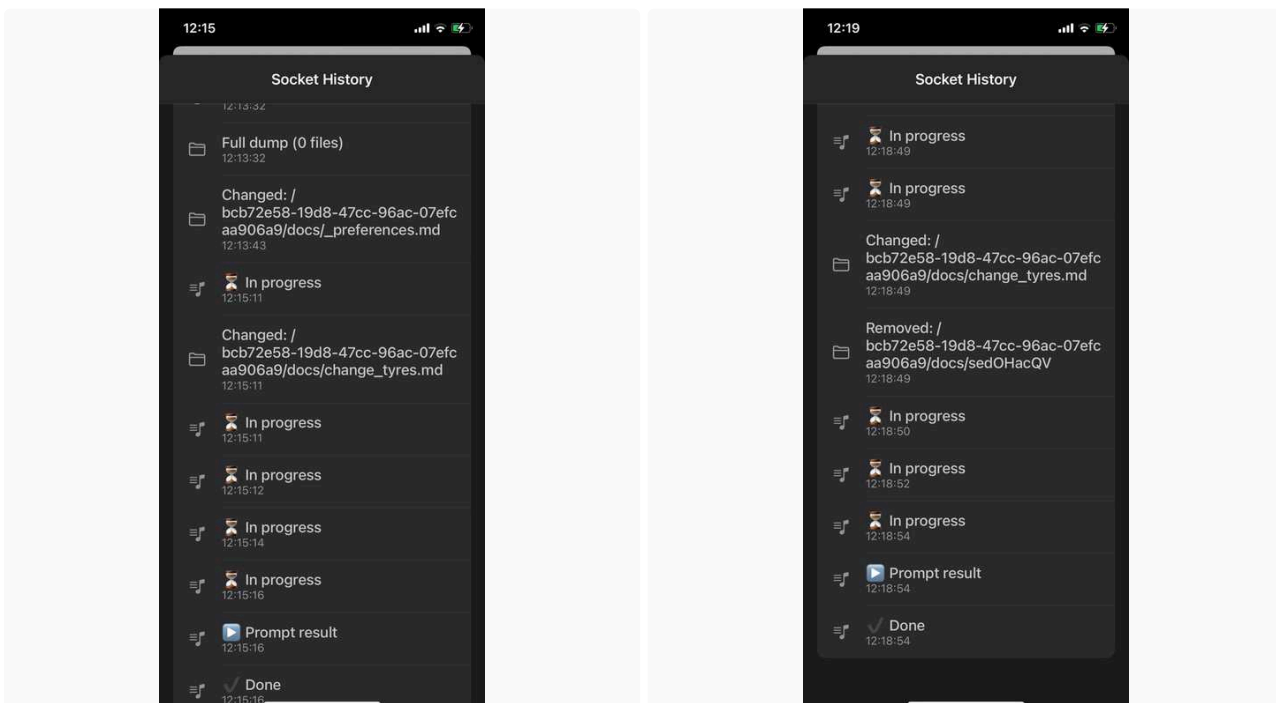
can just sign in now with `bun run auth` (will use `login2.chaps.app`), updated [Readme](#) how to use it

and from then api calls handle session token, token refresh etc under the hood

also updated with Bartek's small evaluator schema change a few weeks ago, so now it is running again e2e

File watcher issue

I built and played with the new `Chaps.iOS` app, but at quick glance file watching seems to work



Prep for Monday workshop

had a long discussion with BalintB mostly about media/file handling in the chap, but also touching on manual edit, profile chap idea etc

started adding some thoughts to [Client Comms Plan](#)

Other

Checked huge Excalidraw PR again as it is getting close to merge ready state

 **[CRWEB-5980] Excalidraw with blob block data**
#5634 · Opened by chris99

Merged Deploy Editor Deploy Windows Sandbox

Auth

So far no major issues have been identified with the new Auth service 🙌

Infra

`login.chaps.app` has been moved over from Clerk auth to our own auth - `login2.chaps.app` will be sunsetted 📅 2025. May 9., Fri morning.

Moved over our Redis for Auth service into Upstash. Key reasons:

- Upstash is fully compatible with the Redis protocol, Redis clients "just work".
- Redis cloud writes append-only logs every 1 second. Upstash on the other hand guarantees durability by persisting every key-value to replicated durable storage, while keeping the working set in memory.
 - We're using Auth as our primary user store. We can't afford to lose profile data.
- Upstash provides pay-as-you-go pricing, no commitments. Perfect for dev instances.
- REST is supported as well. This allows more clients to connect via HTTPS as opposed to going to port 6379.
- Upstash supports any number of read replicas. Auth is read-heavy.

I'm going to be sunsetting the old Redis Cloud instance 📅 2025. May 9., Fri .

Current setup is primary in `us-east-1` , read replica has not been set up. I'm thinking we should have at least an `eu-west` replica in the future.

A number of code changes had to be made to facilitate the change, primarily around how the official node redis client handles commands and how a simpler client like Upstash's does. All-in-all it's been a net positive: after redoing some of the token logic, Redis call count is down about 20%, lots of writes have been batched or committed atomically in one transaction.

Profile

I've been working on setting up the Supervisor → Auth API for Users.

Built the APIs on Auth for Supervisor. It can now authenticate to Auth and pull a users profile. With that, it can get the users email addresses and find a user by username. Emails are important for the email comms channel in Chaps: if a user is logged in via SMS, the auth token does not have any information on the users email, and as such, would not be able to identify who sends an email to a Chap. Because an email can be sent to either your `{userId}...` or `{username}...` address, Chaps need a search by username endpoint.

Auth now contains user profile handling and username search, which solves these issues.

Authentication to Auth service is done the same way as any other client. A JWT token is created and signed with the user ID of the initiating user. The search by username endpoint requires an admin token (response is not tied to a userID, and should not be accessible from any end client), which is created by supervisor.

Client Comms

Quite a long in-depth discussion with Sipi today around decisions we'll make on Monday. Writing my thoughts into a scratchpad now.

Other

I had nothing to do with Pope Leo XIV's election.

↑ Daily Notes - 2025 May

May 8., Thu



Chaps Team

What's going on in the AI world? · The industry seems to be ablaze with everyone anticipating the next acquisition of...

↑ May 8., Thu

Chaps Team

What's going on in the AI world?

The industry seems to be ablaze with everyone anticipating the next acquisition of an agentic coding tool. OpenAI just acquired Windsurf for \$3B (with a \$40M ARR → 75x their ARR!), Cursor raised \$9B at 30x ARR of \$300M. Historically, Microsoft acquired Github for \$7.5B in 2018 at 25x their \$300M ARR. Why is this important? There's one major player in this space who hasn't made a public move yet. Google. Yeah there's [Code Assist](#) - but it's just an IDE extension. They haven't gone full circle. *(Cursor, I'm looking awkwardly at you...)*

But what Google does have, is an exciting new (version of their) model: [Gemini 2.5 Pro](#). There's quite the buzz around it on various channels. If anyone's tried it please let me know!

[Mistral Medium 3](#) is also out, claiming to accomplish around 90% of what Claude's amazing Sonnet 3.7 model can, but at 8x less cost. They also hint at working on an open source "large" model. I haven't tried this one yet either.

OpenAI was also in the news lately for their interesting update to GPT-4o. All of a sudden the world learnt what "sycophancy" means. Essentially, the model was trying to please users at (almost) all costs - including reinforcing negativity, etc. They have an interesting [writeup](#) about what went wrong and how they train & review their models. The evergreen question in AI: how do you measure what a model "feels" like?

Microsoft has [teamed up with Google](#) on some kind of collaboration around MCP and A2A. A2A & MCP might not be dead afterall - if giants start adopting it, the industry could follow.



In some areas - coding in particular - it does feel like an arms race to create larger context windows, higher benchmark scores, faster response times. **Which means it's a really exciting time to be working on the applied side.**

 Peter Sipos

managed to plumb together prefilling the country code based on geolocation, it was quite a fight with Vercel docs (and LLMs), basically took almost the whole day, but finally works 🎉

A brief overview of the complications/time wasters:

- the whole Vercel edge middleware feature is basically originally a NextJS feature. It should theoretically work without framework, but out of like 20 Vercel middleware examples, only one is not NextJS, and it is very simple, and even contradicts the docs.
- also LLMs did a lot of hallucination, generating not working fixes (tried multiple models), so the whole complexity and moving target nature of Vercel features meant they were not really being able to come up with good solutions in this space.
- so in theory Vercel should automatically pick up a root `middleware.js/.ts`, but it just didn't work
- after a lot of attempts, wrong tries and many wrong fix attempts by LLMs, I decided to drop the middleware approach, and instead implement a Vercel Edge function, and instead of the `index.html` already having a country code, make an extra api call to this edge function from the client to get the country
- now the problem is, again after some tries and debugging, is that edge Vercel functions seemed to only work with a new way of configuring project builds (turns out ours was an old pattern), using the new `functions` prop instead of the old `builds` so the `vercel.json` had to be migrated to this new functions syntax
- I was already busy setting and debugging the edge function, when accidentally noticed that the middleware which I also left there started to work. → so it does work, but not with out earlier legacy `builds` approach 🤖
- so resumed the middleware track, but still had a couple of hiccups, for example hallucinated APIs by LLMs that were not actually there (like a `HTMLRewriter` which is only in `CloudFlare` edge)
- also it turned out it is not anymore possible to modify/create a response manually in edge functions, probably due to this limitation [Returning response body in middleware](#), so I ended up using a `cookie` instead - but only as "transport", I immediately delete it on client side when app is loaded and move it to a `meta` tag as originally intended
- a bonus problem was that on page refresh it did not work e2e (middleware was running, but cookie was not set), only on genuine first url navigations in the browser. Turns out on etag-not modified responses, the browser can and does drop cookies, so had to also tweak the middleware to prevent etag caching for this resource.

Anyway now it seems to be working 🎉



feat(init-country): prefill country code based on geolocation

#4 · Opened by Schipy · Merged by Schipy

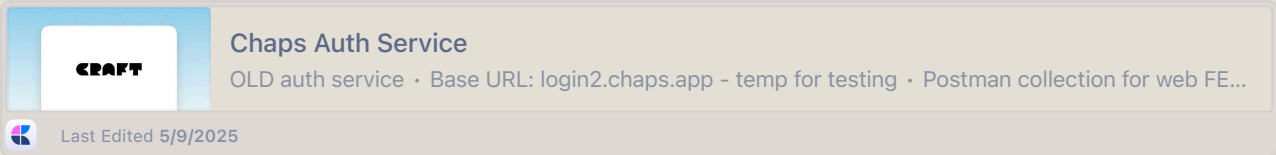
Merged



Next I'm moving on to checking the file watcher not emitting file changes.

Chaps Auth Service

Chaps now has its own authentication service, replacing Clerk for authN and authZ 🎉



I've implemented authentication and authorization on the Chaps API layer with tokens handed out by the new Chaps Auth Service. The Service and API layers have been deployed and are now authenticating requests to the Chaps API.

There were a number of complications when going live. JWT secret keys were encoded slightly differently on the Auth side and the API side, leading to JWT signature errors, and HTTP 401 Unauthorized responses. From what I understand, setting the key via console (`fly secrets set AUTH_JWT_SECRET=xyz...`) and via the web UI results in different encodings - no emojis, special chars, overlong chars, etc. We also ran into a size limit of an app (a Chaplet) on fly.io of 63 characters. Our new user ID format (`cu1_{UUID}`) pushed the names past this limit. They have been replaced to be `chaplet-prod-user--{UUID}` .

For more design details, see my daily note from yesterday.

↑ Daily Notes - 2025 May

May 7., Wed



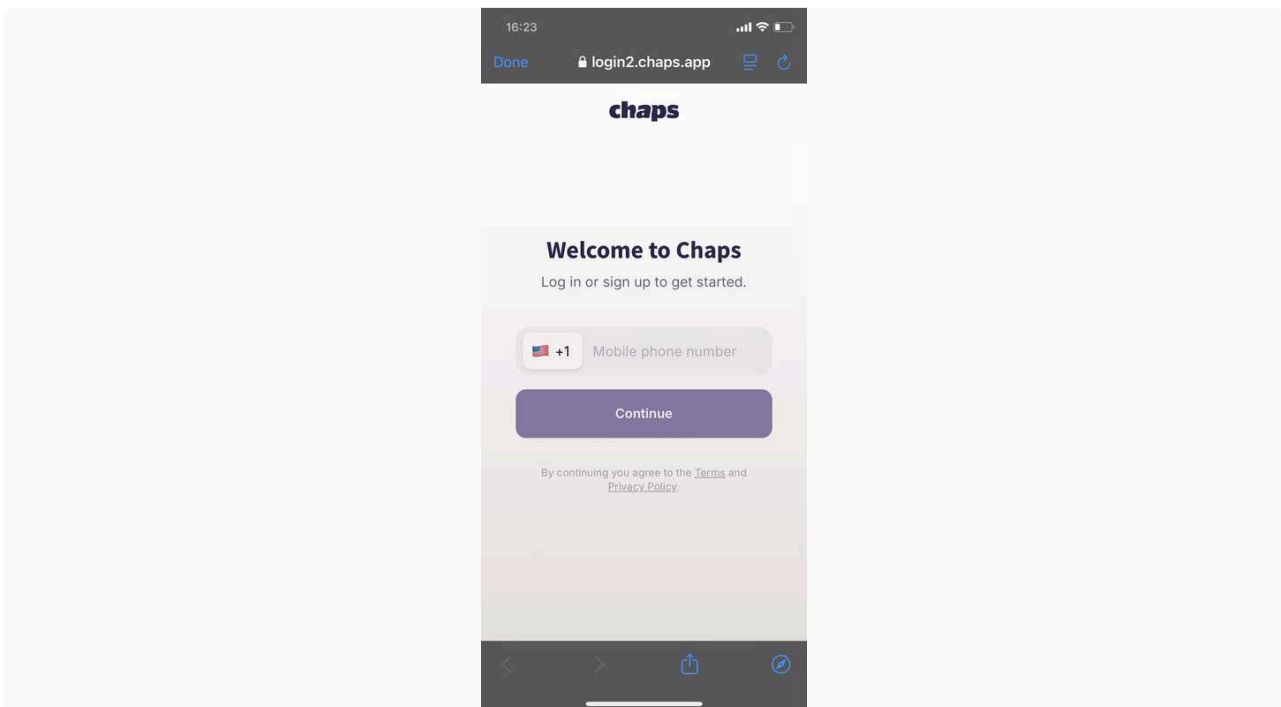
Chaps Team

Peter Sipos · SMS login, latest flow, with all improvements

↑ May 7., Wed

Chaps Team

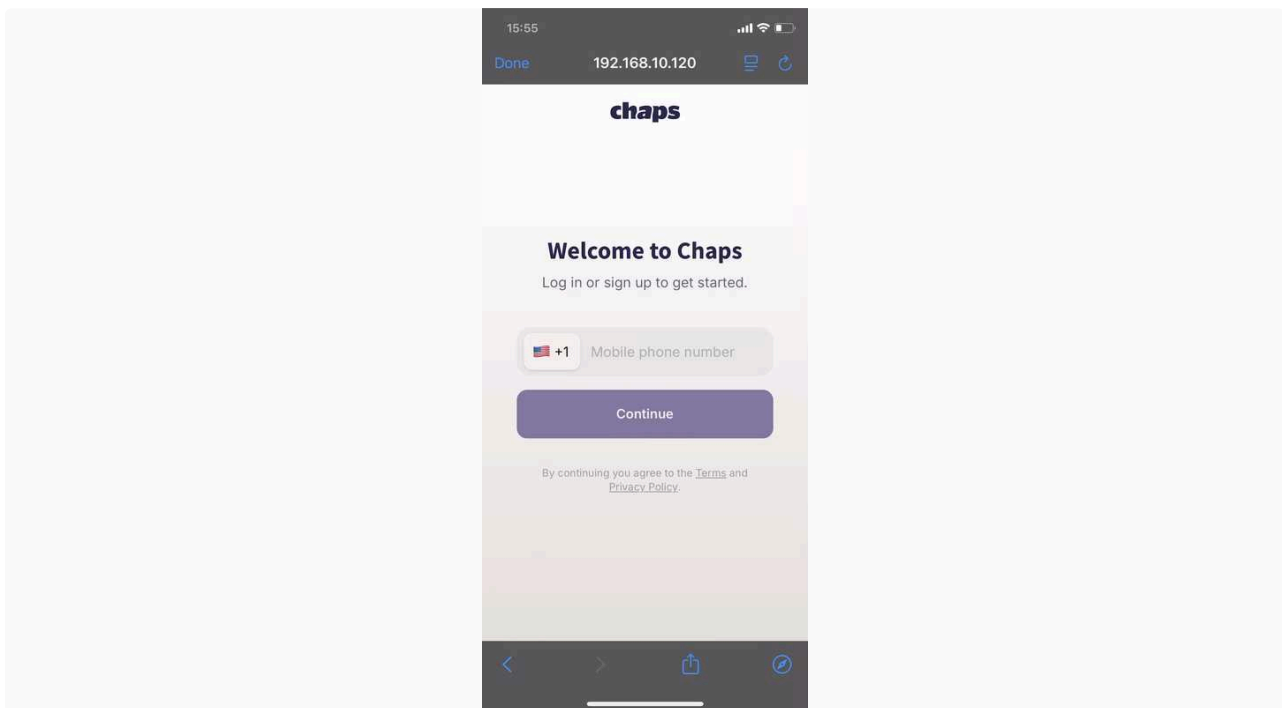
SMS login, latest flow, with all improvements
(available on <https://login2.chaps.app/#phone>)



Details of improvements & fixes

- added cross-fade between screens
- disabled Continue button while phone number is invalid (this is a [lib](#) ported from Google's Android phonenumber lib, super mainstream & big, it must have reliable validation)
- actual backend api wired in
- also added a test mode with `?test` which uses the mock api and disables phone number validation (yes, it's also possible to nicely load the screens with `#` for testing, but this also allows to test the whole flow)
- added user feedback when verify is in progress (disabled OTP input - doubles as user feedback as it should be disabled anyway)
- error handling when login attempt call fails
- the previous two came with a bit of refactor and improved bookkeeping of last backend request/response we had
- tested on real mobile device & some mobile layout fixes
- tested & autofill from SMS works on iOS
- allow quick manual input of country code on mobile sizes

This is an interesting one. On desktop the country dropdown is nice, as I can type eg "HU" and it jumps to Hungary. But on mobile I found it very slow to use, as I have to scroll a multiple hundred dropdown to find my country 📌

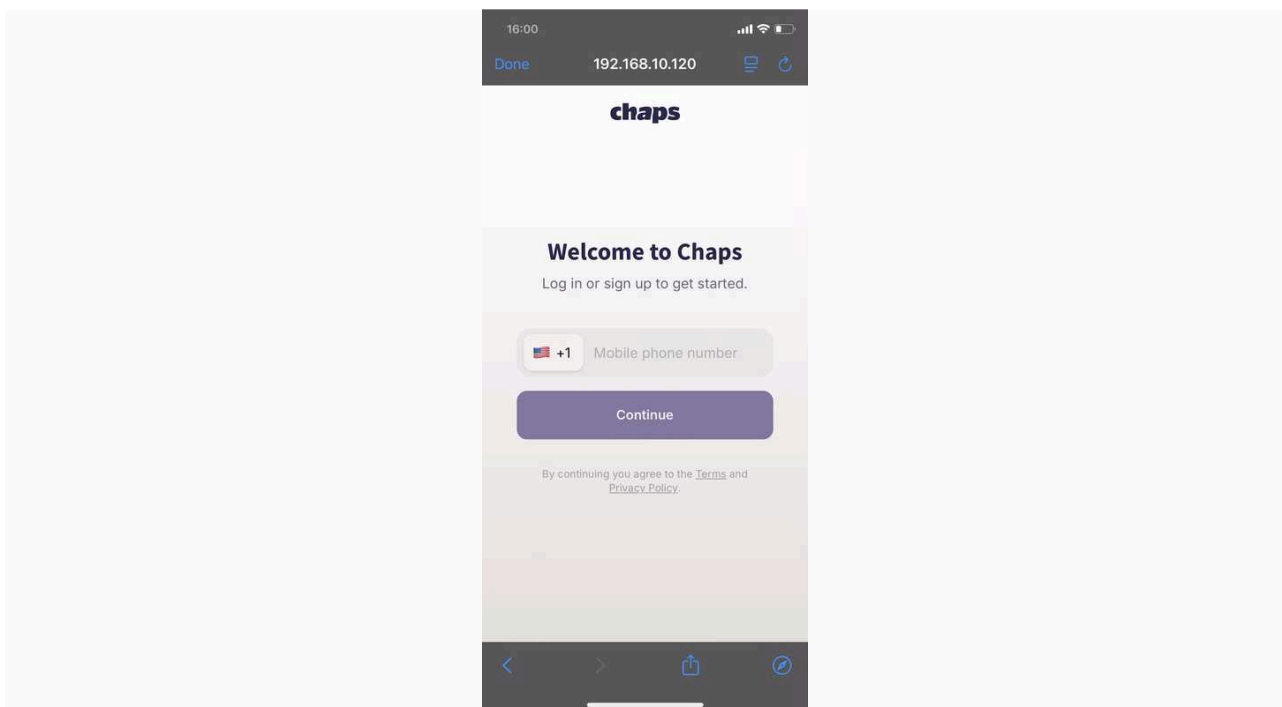


I had a few solutions in mind, eg I think we should pass country-detection info to the front-end somehow:

<https://vercel.com/guides/geo-ip-headers-geolocation-vercel-functions>

But in any case, even if we have prefilled country, I wanted to see if this can be improved, and added now a variant where

- on desktop screen sizes, the full badge pulls up the dropdown
- on mobile size, the flag pulls up the dropdown, but tapping on the number gives you text edit to quickly type in the country code



What's left - not much for now, except if we want to add the user name entry or other feature

- adding/testing the actual deeplink to open
- Prefill country code based on geolocation (maxi smooth login to minimize churn)
- test on iPad size
- ~~not now: test autofill from SMS on non Safari mobile browsers, for Chrome either works or needs maybe [WebOTP API](#)?~~
- once session tokens actually start working (on chaps apis), then look into how auth can be updated for cli evals tool

 2025. May 7., Wed

what is the next highest prio thing?

if nothing else, then can resume looking into Chaps API improvements, unless there's much feedback/requests coming in for login screen

Auth service

Working with Sipi this morning, Auth service has been deployed fully (Frontend, Backend API) to Vercel under a single application. This means we are now able to fully test the login flow E2E. Since all auth API calls are stateless, we are using Vercel functions, deployed on the edge. I set up the domain, current login page can be tested at <https://login2.chaps.app>.

For integration, I created [Postman docs](#) for the current API calls to auth, with more coming as we finalize the user profile endpoints. Auth can be run locally as well for fast dev iteration. Some minor issues have been ironed out.

To productionize the auth service, I set up our production Redis instance. Current setup is

- Location `us-east-1`. This will give users in the US the lowest latency.
- Persistence with an append-only log written every second. Backups every 24h, stored on Redis cloud, 7 day retention. We can move backups to our own S3 buckets for longer retention if need be.
- Size is 250MB. A user takes up roughly
 - ~250 bytes for an auth token, 100 bytes for token usage counters
 - ~400 bytes for a session token (which will not be stored, I'm only using it for debugging)
 - ~2k tops for the user profile JSON
 - This means an average user with an average number of logins (let's say 2) will take up ~3k bytes.**With the current setup we can store ~80k users**, with everything in memory.
- Cost will be 6USD/month

I've built a very simple user profile store, which uses Redis JSON objects. Schema is parsed with Zod, so it's pretty flexible and future-proof.

Scaffolding is in place for username management as well, with lookups by username (for Postmark emails).

token		70%	19
tokenset		4%	1
SORTED SET	u_45b984b0-b0ad-4d22-9a76-c3df8d023ed8	No limit	184 B
tokenused		19%	5
STRING	5506bc9f964be3f413ee2ee903866921fb2033a58104afbafdb580b7f42d5fdc	29 d	136 B
STRING	5d18e0bc44202b5b3103b8a78f1bb5825af5d48b80c64d91100e7e4ea2c354b8	29 d	136 B
STRING	8caea7799ef68ae1c31caf84dd6427440f978ecbda76bbca2558d8fa62eeac1d	29 d	136 B
STRING	dda154847fbc5e6b078e1fc11b2646fde9c3aad4aec790c0108cb97739ff8373	29 d	136 B
STRING	fd3b6ec05590d9d2e4180494b147ab95b873d7b094b1bb64fc98c7c163b2e428	29 d	136 B
login		4%	1
phone		4%	1
HASH	+36304394545	No limit	152 B

Repo <https://github.com/lukilabs/chaps-auth>

We're able to invalidate a users tokens (long, short), and tokens are immediately deleted when a user is deleted. We're using Redis sorted sets for this. <https://github.com/lukilabs/chaps-auth/compare/token-handling-2?expand=1>

I'm now working on proper token handling (authN, authZ) for chaplet supervisor ([branch here](#)) and the chaplet API itself. This should be fully ready tomorrow - we can start E2E testing with the iOS client.

Since we are not using Clerk anymore, user profile information will be provided by the auth service. For the supervisor or chaplet to connect to the auth service for user info, it will auth with a JWT signed with the same shared secret. Building endpoints for searching by username as well.

↑ Daily Notes - 2025 May

May 6., Tue



Chaps

Sorry folks, another day without an AI roundup. I'll be writing one tomorrow, today was another large push to get a c...

↑ May 6., Tue

Chaps

Sorry folks, another day without an AI roundup. I'll be writing one tomorrow, today was another large push to get a custom auth layer working for Chaps.

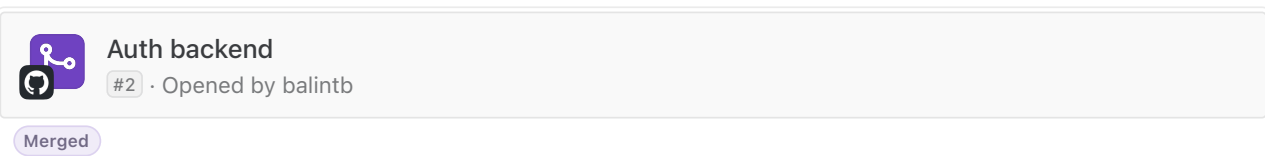
Chaps custom auth

Synced with Sipi to agree on the login flow. Set up Twilio account for Chaps, which we will be using for sending SMS (TOTP for login/signup).

After that I was heads-down today implementing Chaps auth.

Implemented and working locally

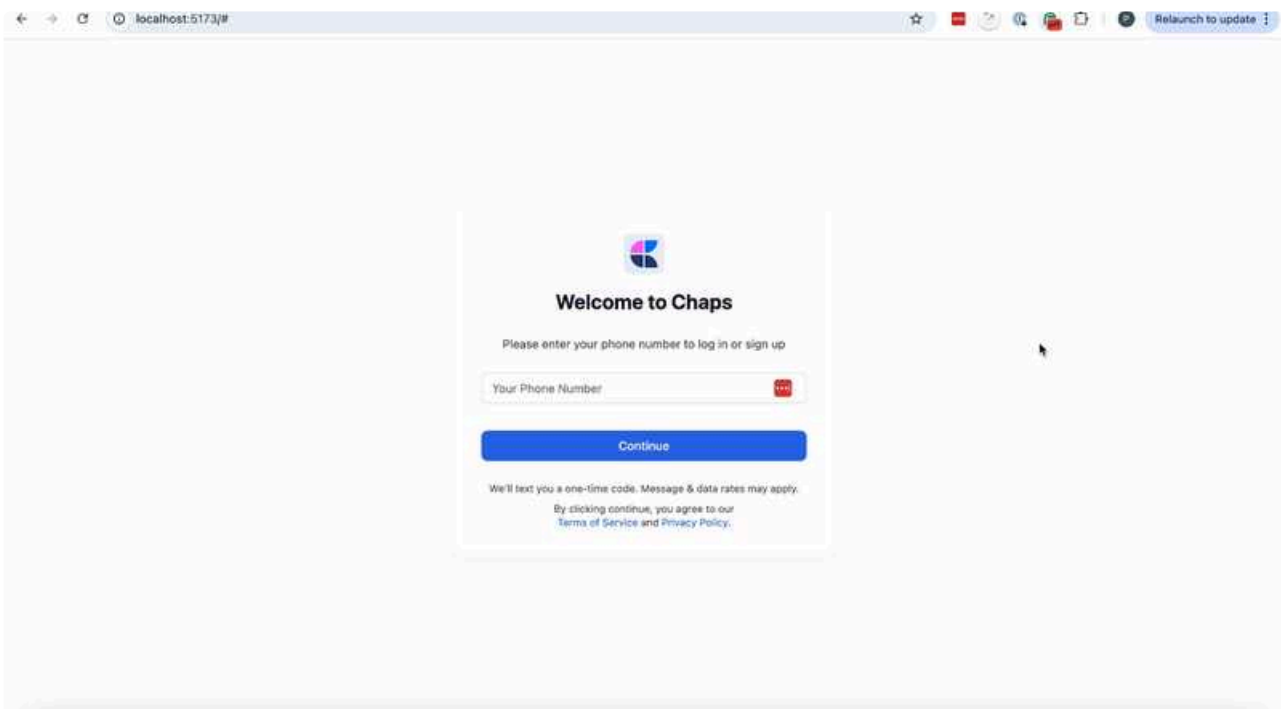
- **Twilio OTP SMS handling.** Login / signup works similar to Craft. The user enters a phone number (later on email will also be possible), receives an SMS with an OTP, enters that code and is logged in. Login attempts are rate-limited (configurable). After successful verification of the OTP, the client is issued an auth token.
- **Auth token handling.** After receiving an auth token, the (iOS) client makes a call to `/refresh` and receives a session token. The session token will be used to validate the API calls made to chaplet APIs. Auth tokens are very long-lived (30 days, configurable), and are limited to 10 uses (configurable - this should possibly be only 1-2). After N uses, they are rotated to guard against compromised clients, and on `/refresh` the client will receive a new auth token along with the session token.
- **Session token handling.** Session tokens are the primary authentication key on the API layer. They are issued by the auth service, have a much smaller TTL (24 hours, configurable), and are not checked online, but checked offline by validating their JWT signatures (shared secret). They are valid, signed JWTs, with the user information embedded into them. These tokens contain a valid signature, which is a refreshing change from Clerk.
- **Email.** Scaffolding is in place to enable email TOTP as well, not just SMS.
- **Redis.** Auth tokens are stored hashed as a Redis Hash. TTLs are used for deleting keys: TOTP validations are rate-limited by setting locks on the identifier (SMS or email). When auth tokens are rotated, there's a 1 minute (configurable) grace period while the client can still use the old token (if, for example, there's a network issue). After the TTL old tokens are deleted.



These flows have been tested locally, but some work still needs to be done to deploy online (mainly around prod Redis infra, Vercel deploys of both FE and API, secrets handling). Postman docs also need to be written - but for now, all request/response Zod schemas are [here](#).

After fusing this with Sipi's frontend, I'll move on to implement these tokens on the chaplet API layer (offline verification with JWT signatures), then on to basic user profile handling.

focused worked on Chaps auth frontend all day



auth-fe

Directory

apps/auth-fe



lukilabs / chaps-auth

Branch main

I went with PNP, Vite, Tailwind + shadcn/ui

PNPM

I went with pnpm, because besides being fast, it is very robust and well known, and should also well work together if we want to work with monorepo workspaces (eg Turborepo), to deploy the BE and FE together etc.

Vite

Good and established choice now, as we agreed before.

Tailwind

For easy customization and tweaking

shadcn/ui

I chose now shadcn/ui over Tailwind Catalyst UIKit (not very hard to swap at this point), because

- both are based on ejecting the components into our codebase, so are highly/fully customizable

- but Shadcn has much more components, including an OTP input component that we need, and more community/traction
- and also seemed like if we don't want the default Tailwind look and feel, then the UIKit does not add much

Framer

tried Framer for animation, because it should have a well established and easy to use API (footprint may be unnecessary given the small scope, but should be ok at this point). However I'm not satisfied with the transition between states yet. Actually this is the only area where I kindof wasted some time and spent over an hour trying to fix up the framer animation (mostly with Cursor, as I'm not experienced in Framer much) What I wanted to achieve is a proper morph (or ~FLIP) animation, where we cross fade content of sections, while its bounding box animates the height smoothly between them. I can achieve this with old react transition groups and size observation, but that would also be quite convoluted and wanted to something more modern and tweakable-by-design. I still think it's likely achievable in Framer (maybe along the lines of the [popLayout mode](#)), but did not want to lose more time on this now.

What's left

- wiring in the actual backend API
- there's no user feedback while the OTP is validated (to show something is happening...)
- opening deeplink automatically
- design polish
- animation polish (rewrite?)
- there are OTP input ux gaps I think (eg I could not paste the code, only if the OTP input is in focus)
- need to add & test mobile OTP autofill hooks (Android SMS Retriever API / iOS AutoFill)
- need to test mobile/iPad layout

Sounds like one more day on polishing all the details (don't need to do all right now, but somewhere down the line)

↑ Daily Notes - 2025 May

May 5., Mon



Chaps Team

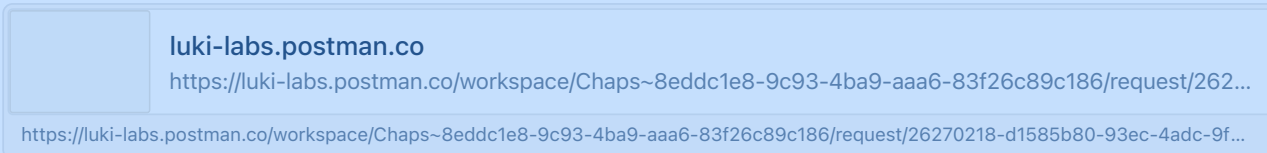
Peter Sipos · History API · added a GET `/api/chaps/:chapId/history`, which returns the conversation history · luki-labs...

↑ May 5., Mon

Chaps Team

History API

added a `GET /api/chaps/:chapId/history`, which returns the conversation history



interface/communicator vs agent exchange

modified the conversation history to contain the actual message exchange on the interface (eg communicator) and it contains the internal agent message as a property (which is the message exchange on the main entry point agent. We may hide this eventually from api, but for now it is returned as well).

The interface/communicator and agent exchanges may differ in several ways:

- input example: the actual user input may be audio vs the internal agent message is the transcribed string
- input example: we have not implemented in-chat widget behaviors yet, but the actual interface user message may refer to the communicator message (eg "user pressed button named "remind me" with property "2025-04-30T16:00"), which will be translated to internal agent just as "remind me at 2025-04-30T16:00")
- output example: user facing entry point messages now go through communicator, so the actual response on interface is the communicator response

multi-modality

will be supported by upgrading `content: string` type to `content: string | Array(TextContent | ImageContent)`, similarly as how OpenAI API does it. (the plain string content is just a shorthand for a single text type content item)

different entry points, different communication channels, different users

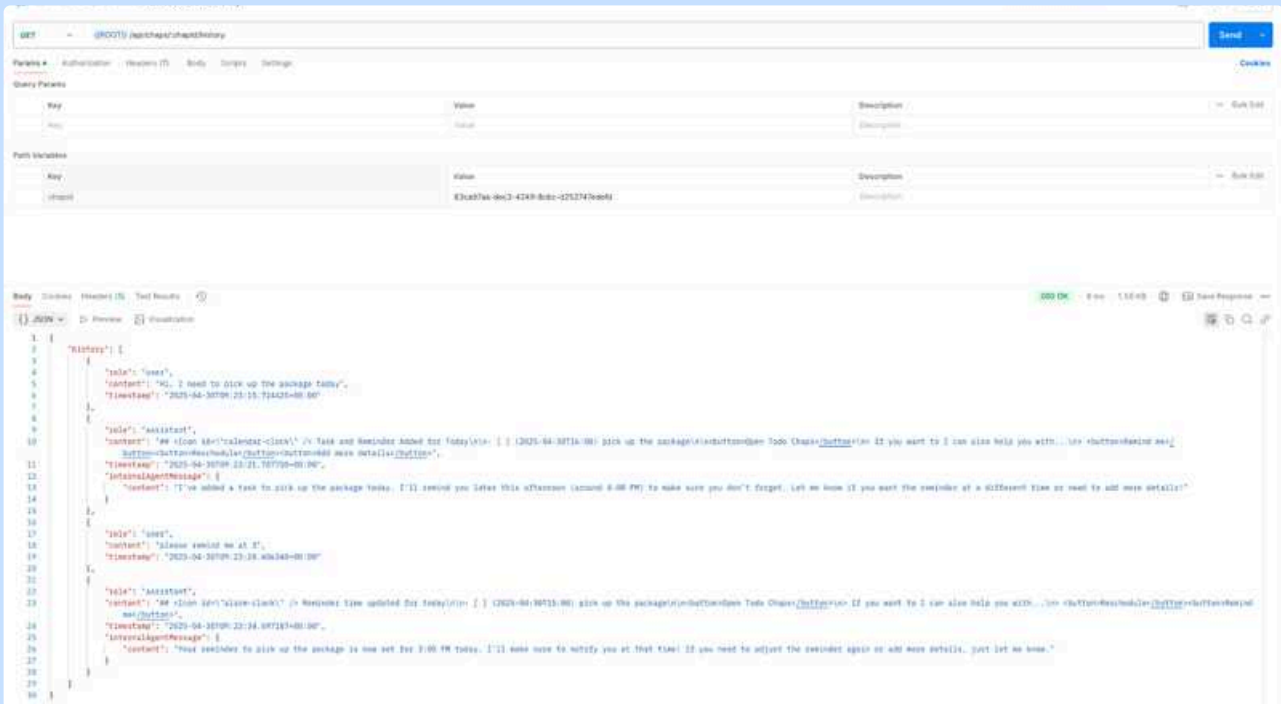
my idea is they will be supported by actually maintaining multiple conversation histories, a separate message thread

- per entry point (eg main chat, email, wake ups in current todo app)
- per user (if sharing is supported later, and someone else is chatting with your chap)
- per thread (if multiple threads will be supported per chap, maybe email entry works like this by default)
- manual file edits will likely be an entry point in this interpretation, so will have their own history

client will be able to query different conversations tracked and can decide whether to present them separate or merged etc. It can also specify which conversations to include in a user request as context. (eg can issue user message where the agent gets not just that particular thread, but also email exchanges in between)

this is not yet supported, but for managing email entry point, will likely need to be tackled soon

current api endpoint in action



merged Final prompt response

Issue: currently the final response (~what we also save in history and you get on history API) is not emitted in a nice way. We're following a result stream pattern (like when streaming is on on OpenAI api), and we send prompt progress events from agent executions. At the moment the final response can be obtained by checking the following type chain on the object

```
"orchestra" > "prompt" > "progress" > "final_response"
```

and take the **second** such message (or filter by `agent_id`) which will be the communicator final result (as we have two execution, one is the actual todo agent, second is the communicator). There's also some traces in code of a final prompt extraction, but it is also doubled and it is only used for internal clients etc, needs cleanup.

This is far from ideal.

Goal: have a clean, single final response message, that also matches the content of what's saved in history as expected.

[WiP] feat(agent-communicaton): cleaner final prompt-result message for executions

Merged

in the new setup there's a single

```
"orchestra" > "prompt" > "prompt-result"
```

message emitted that contains the final result (same as that saved in conversation history). The CLI tool has been updated to show an example for processing. Eval tool has not been updated to use this.

New Auth infra

also did some exploration around new in-house auth infra, and sent the input to BalintB

Balint Bartfai

Today was debugging Clerk authentication, especially around session tokens.

I managed to unblock Balint, but with one huge caveat: the JWT signature is not being checked. For some reason, Clerk issues session tokens with an invalid JWT signature (verified with their own libraries, with the correct secret key(s), the correct JWT pubkeys). So the decision was made to ditch Clerk and build our own authentication.

I was trying to get a token for the supervisor to redeploy a chaplet. Ended up having to delete the app and doing manually via cli. Auth has been removed temporarily from /redeploy until we sort out what the heck's going on with how to generate a good token to call the supervisor.

I started drafting the auth system. I'll be working on this Monday, *aiming* for a working proto implementation by end-of-week.

The high level plan

- Issue TOTP-s via SMS to users. The TOTP grants them (the client) a long-lived token (around 30 days)
- Long-lived tokens are stored on the auth service side (hashed), and periodically rotated after X number of uses, so as to mitigate the token being compromised via the client
- The client requests a session token (JWT) using their long-lived token
- This session token can be used to call the API
- Session tokens are verified on the API side without the need for central checks (via signatures)
- When session tokens expire, the client generates a new session token via their long-lived token. After X uses, long-lived tokens are rotated, the client gets a new one via the same call
- Rinse & repeat

Infrastructure plan

- TBD, but it would be nice to scale this out on the edge
 - *An example* that comes to mind is Cloudflare Workers KV for TOTP-s, D2 for long-lived tokens and user data

Frontend

- Built by our own world-class web team